

LEHRMITTEL • INFORMATIKER EFZ • 2. LEHRJAHR

Adressverwaltung Tutorial WebApp v2.1

Full-Stack-Applikation mit Next.js, C# OData und PostgreSQL

Next.js 14

C# ASP.NET Core 8

OData v4

PostgreSQL 16

Docker Compose

NextAuth.js

JWT

Version 2.1

September 2026

Informatiker EFZ

Inhaltsverzeichnis

1. Projektübersicht	3	6.2 Warum JWT statt Sessions?	
1.1 Was ist die Adressverwaltung?	3	6.3 JWT im Backend konfigurieren	
1.2 Lernziele	4	6.4 JWT ausstellen (AuthController)	
1.3 Technologien	4	6.5 Endpunkte schützen ([Authorize])	
1.4 Architektur	5	6.6 JWT im Frontend speichern	
2. Projektstruktur	7	6.7 API-Aufrufe mit JWT (api.ts)	
2.1 Verzeichnisübersicht	7	6.8 Login-Ablauf Schritt für Schritt	
2.2 Warum diese Struktur?	8	6.9 Sicherheitshinweise	
3. Entwicklungsumgebung	10	7. Frontend – Next.js 14	
3.1 Voraussetzungen	10	7.1 App-Router-Struktur	
3.2 Docker Compose starten	11	7.2 Routen-Schutz (middleware.ts)	
3.3 Dienste und Ports	12	7.3 PLZ-Autocomplete	
3.4 Umgebungsvariablen	12	7.4 React Best Practices	
3.5 Standard-Login	14	8. Clean Code – Massnahmen	
4. Datenbankmodell	15	8.1 Backend-Befunde (B-01 bis B-08)	
4.1 AuditableEntity Basisklasse	15	8.2 Frontend-Befunde (F-01 bis F-06)	
4.2 Entitäten-Übersicht	18	9. Docker Compose	
4.3 EnsureCreated vs Migrations	20	9.1 Dienste-Konfiguration	
4.4 DbSeeder (SRP)	21	9.2 Wichtige Befehle	
5. Backend – ASP.NET Core 8	23	10. Hilfsskripte (scripts/)	
5.1 Program.cs Dienstregistrierung	23	10.1 PLZ-Import (import_cities.py)	
5.2 Middleware-Reihenfolge	26	10.2 Benutzer anlegen (create_user.sh)	
5.3 OData – Grundlagen		10.3 Passwort zurücksetzen	
5.4 Generischer ODataCrudController		11. Häufige Fehler & Lösungen	
6. JWT-Authentifizierung (NEU)		11.1 SqlState: 28P01 (Passwort)	
6.1 Was ist ein JWT?		11.2 401 Unauthorized bei API-Aufruf	

KAPITEL 1

1. Projektübersicht

Lernziele dieses Kapitels

- Die fachliche Zielsetzung und den Funktionsumfang der Adressverwaltungs-WebApp verstehen
- Die Trennung von Frontend, Backend und Datenbank im Full-Stack-Kontext begreifen
- Den gesamten Technologie-Stack mit den jeweiligen Aufgaben der Frameworks einordnen
- Das Kommunikationsmuster zwischen Browser, Next.js Server, ASP.NET Core API und PostgreSQL nachvollziehen

1.1 Was ist die Adressverwaltung?

Die **Adressverwaltung** ist eine praxisnahe Full-Stack-Webapplikation für den geschäftlichen Einsatz zur zentralen Erfassung, Pflege und Verwaltung von Kontaktdaten und Firmenadressen. In modernen Unternehmen bilden saubere Adressbestände das Fundament für CRM-Systeme, Fakturierung, Logistik und Kundensupport. Eine mangelhafte Adressqualität verursacht im Geschäftsalltag erhebliche Kosten durch unzustellbare Postsendungen, falsche Rechnungsadressen und Medienbrüche.

Die hier entwickelte WebApp löst diese Herausforderungen durch eine modulare, entkoppelte Drei-Schichten-Architektur:

- **Sichere Authentifizierung:** Benutzer melden sich über ein modernes Web-Portal an. Passwörter werden nach dem aktuellen kryptografischen Standard mit PBKDF2 und Salt gehasht. Die Sitzung wird zustandslos über JSON Web Tokens (JWT) gesteuert.
- **Vollständiges Adressmanagement (CRUD):** Adressen können erstellt (Create), gelesen und durchsucht (Read), geändert (Update) sowie gelöscht (Delete) werden.
- **Intelligente PLZ-Wertehilfe (Autocomplete):** Bereits während der Eingabe der Postleitzahl schlägt das Frontend in Echtzeit passende Schweizer Ortschaften vor. Die Datenbasis basiert auf dem amtlichen Ortschaftsverzeichnis der Schweizerischen Eidgenossenschaft (swisstopo / Schweizerische Post) mit über 4'000 Ortschaften.

- **Lückenlose Nachvollziehbarkeit (Audit-Trail):** Jeder Datensatz speichert automatisch Erstellungs- und Änderungszeitpunkt sowie den ausführenden Benutzer und einen Gültigkeitszeitraum (DateFrom / DateTo).
- **E-Mail-Benachrichtigung:** Bei der Neuanlage einer Adresse wird automatisch eine formatierte Benachrichtigung an eine konfigurierbare E-Mail-Adresse versendet.

1.2 Lernziele für Informatiker EFZ

Dieses Tutorial richtet sich gezielt an **Lernende im 2. Lehrjahr der Fachrichtung Applikationsentwicklung (Informatiker/in EFZ)**. Es verbindet theoretisches Grundlagenwissen aus den überbetrieblichen Kursen (üK) und der Berufsfachschule mit praktischen Best Practices aus der Software-Industrie.

Kompetenzbereiche nach Bildungsplan

Das Projekt deckt wesentliche Handlungskompetenzbereiche der Schweizer Informatik-Grundbildung ab:

- **Modul 165 / 293 (Webauftritt / Web-Frontends):** Erstellen responsiver Benutzeroberflächen mit React und Next.js, State Management, Controlled Inputs und Client-Side Routing.
- **Modul 320 / 322 (Objektorientiert programmieren / Entwurfsmuster):** C# .NET 8, SOLID-Prinzipien, Single Responsibility, DRY (Don't Repeat Yourself), Generics und Dependency Injection.
- **Modul 164 / 183 (Datenbanken / Sicherheit):** Relationales Datenmodell in PostgreSQL, O/R-Mapping mit Entity Framework Core, Migrationen vs. Schema-Generierung, SQL-Constraints und Transaktionen.
- **Modul 184 / 187 (Software-Architektur & Security):** REST/OData Schnittstellendesign, JWT-Token-Validierung, CORS-Policies, Hashing vs. Verschlüsselung und Timing-Attack-Prävention.

1.3 Technologien im Detail

Für die Realisierung setzen wir auf einen zukunftssicheren Stack aus quelloffenen, industrieweit etablierten Technologien. Die nachfolgende Tabelle gibt einen strukturierten Überblick über die einzelnen Schichten:

Tabelle 1.1 Technologie-Stack der Adressverwaltungs-WebApp v2.1

Schicht / Bereich	Technologie	Version	Aufgabe im Projekt
Frontend-Framework	Next.js (App Router)	14.2+	Server Components, SSR (Server-Side Rendering) und Client Interactivity
Programmiersprache Frontend	TypeScript	5.3+	Statische Typprüfung, Intellisense und Typverträge für API-Modelle

Frontend-Auth	NextAuth.js	4.24+	Session-Management, JWT-Cookie-Handling, Route Protection
Styling / UI	Tailwind CSS	3.4+	Utility-First CSS für konsistentes, responsives Design ohne globale CSS-Kollisionen
Backend-Framework	ASP.NET Core Web API	8.0 LTS	High-Performance REST-Dienste, Dependency Injection, Middleware-Pipeline
API-Protokoll	Microsoft.AspNetCore.OData	8.2+	Standardisierte URL-Abfragesprache (\$filter, \$select, \$orderby, \$top)
O/R-Mapper	Entity Framework Core	8.0	Code-First Datenzugriff, LINQ-Abfragen, automatischer Audit-Trail
Authentifizierung	JWT Bearer (HMAC-SHA256)	8.0	Zustandslose Token-Validierung bei jedem Backend-Request
Datenbank	PostgreSQL	16.1 Alpine	ACID-konforme relationale Datenbank, JSONB-Unterstützung, Stabilität
Datenbanktreiber	Npgsql.EntityFrameworkCore	8.0	PostgreSQL-Provider für Entity Framework Core mit Typtreue
Infrastruktur	Docker Compose	v2.24+	Reproduzierbare Container-Orchestrierung für Entwicklung und Test
Import-Tools	Python + psycopg2	3.11+	Batch-Import amtlicher PLZ-Daten aus Schweizer CSV-Dateien

1.4 Architektur-Überblick

Ein grundlegendes Prinzip moderner Software-Entwicklung ist die **Separation of Concerns** (Trennung von Zuständigkeiten). Das Frontend übernimmt ausschliesslich die Präsentationslogik und Benutzerinteraktion, während das Backend für Validierung, Autorisierung, Geschäftsregeln und Datenpersistenz zuständig ist.

```

+-----+ |
BENUTZER / WEB-BROWSER | | (Google Chrome, Mozilla Firefox) | +-----+
+-----+ | Port 3000 (HTTP/HTML) |
NextAuth Session-Cookie ▼ +-----+
+-----+ | FRONTEND: NEXT.JS 14 (Node.js) | | - App Router: app/page.tsx,
app/login/page.tsx, app/adressen/page.tsx | | - React Server Components (RSC) für
datenintensive Ansichten | | - React Client Components ("use client") für Formulare &
Autocomplete | | - NextAuth.js Middleware zur Absicherung aller internen Pfade | | -
lib/api.ts: Zentraler API-Client hängt Bearer <JWT> an jeden Backend-Aufruf | +-----+
+-----+ | Port 5000
(intern 8080) | REST / OData v4 JSON HTTP Header: Authorization: Bearer <JWT> ▼ +-----+
+-----+ | BACKEND:

```

```

C# ASP.NET CORE 8 WEB API | | - Middleware Pipeline: CORS -> Routing ->
UseAuthentication -> UseAuthorization | | - AuthController: Login, PBKDF2 Password
Verification, JWT Generierung | | - ODataCrudController<T>: Generische Basis für
standardisiertes CRUD | | - AdressenController: Adressverwaltung mit Audit & E-Mail-
Trigger | | - CitiesController: Autocomplete-Suche mit startswith() / $filter | | -
SettingsController: Globale Systemkonfiguration [Authorize(Roles="Admin")] | | -
Services: INotificationService, IEmailService, DbSeeder | | - Data: AdresseDbContext mit
SaveChanges-Override für automatisches Auditing | +-----+
-----+ | Port 5432 | PostgreSQL Wire Protocol ▼
+-----+ |
DATENBANK: POSTGRESQL 16 (DOCKER) | | - Tabellen: "Adressen", "Cities", "Users",
"Settings" | | - Persistentes Docker-Volume: pgdata | | - Datentypen: VARCHAR, TIMESTAMP
WITHOUT TIME ZONE, SERIAL PRIMARY KEY | +-----+
-----+

```

Kommunikationsfluss im Detail

Anhand einer typischen Interaktion – der Erfassung einer neuen Adresse – lässt sich der Ablauf präzise nachvollziehen:

1. **Authentifizierung:** Der Benutzer meldet sich unter `/login` an. Das Backend validiert das Kennwort und signiert ein JWT. NextAuth speichert dieses Token sicher in einem verschlüsselten Cookie.
2. **Formulareingabe:** Der Benutzer tippt im Adressformular die Ziffern `80` in das PLZ-Feld ein. Ein Debounce-Timer wartet 300 ms ab und ruft `/odata/Cities?$filter=startswith(postalCode, '80')` auf.
3. **Schnittstellenaufruf:** Beim Klick auf «Speichern» sendet die Next.js-App einen `POST /odata/Adressen` mit den Adressdaten und dem HTTP-Header `Authorization: Bearer <Token>`.
4. **Validierung & Audit im Backend:** Die JWT-Middleware prüft kryptografisch die Signatur des Tokens. Die `AdresseDbContext`-Klasse ermittelt den aktuellen Benutzer aus dem `HttpContext` und befüllt `CreatedBy` und `CreateDate` vollautomatisch.
5. **Persistenz:** Npgsql generiert ein parametrisiertes SQL-`INSERT`-Statement und schreibt den Datensatz atomar in PostgreSQL.
6. **Benachrichtigung:** Nach erfolgreichem Commit sendet der entkoppelte `INotificationService` asynchron eine E-Mail an die in den Einstellungen hinterlegte Administratoradresse.

2. Projektstruktur

Lernziele dieses Kapitels

- Die professionelle Ordnerstruktur eines Enterprise-fähigen Full-Stack-Projekts verstehen
- Den Zweck der Trennung in `backend/`, `frontend/`, `scripts/`, `documentation/` und `archive/` begreifen
- Verstehen, wie Entwicklungs-Artefakte versioniert und reproduzierbar abgelegt werden
- Clean Architecture-Grundlagen in der Organisation von C#- und Next.js-Projekten anwenden

2.1 Verzeichnisübersicht

Ein aufgeräumtes und vorhersehbares Projektverzeichnis ist die Grundvoraussetzung für erfolgreiche Teamarbeit, reibungslose CI/CD-Pipelines und schnelles Onboarding neuer Mitarbeitender. Im Stammverzeichnis `adressverwaltung/` befinden sich sämtliche Teilprojekte, Skripte, Dokumentationen und Orchestrierungsdateien:

```

adressverwaltung/ └── backend/ │ └── AdressverwaltungApi/ │ └── Controllers/ │ │ └──
AuthController.cs// Login, Registrierung & JWT-Ausstellung │ │ └── ODataCrudController.cs//
Generische Basisklasse für alle CRUD-Operationen │ │ └── AdressenController.cs// Endpunkte für
Adressverwaltung & Benachrichtigung │ │ └── CitiesController.cs// OData-Wertehilfe für Schweizer
Postleitzahlen │ │ └── SettingsController.cs// Systemeinstellungen (Rollenbasiert: Admin) │ └── Data/ │
└── AdresseDbContext.cs// EF Core DbContext, SaveChanges-Audit & ModelBuilder │ └── Dtos/ │ │
└── AuthDtos.cs// LoginRequest, RegisterRequest, LoginResponse │ │ └── SettingsDtos.cs//
SettingsUpdateRequest │ └── Models/ │ │ └── AuditableEntity.cs// Abstrakte Basisklasse mit
CreateDate, CreatedBy etc. │ │ └── Adresse.cs// Entität für Personenadressen mit DataAnnotations │ │
└── City.cs// Schweizer Ortschafts- und PLZ-Stammdaten │ │ └── User.cs// Benutzerkonto mit PBKDF2
PasswordHash │ │ └── Settings.cs// Globale Konfiguration (z. B. NotificationEmail) │ └── Services/ │
└── IEmailService.cs// Abstraktion für Mailversand (austauschbar) │ │ └── EmailService.cs// SMTP-
Implementierung mit MailKit / Smtplib │ │ └── INotificationService.cs// Fachliche
Benachrichtigungslogik (SRP) │ │ └── EmailNotificationService.cs// E-Mail-Notification bei
Adresserfassung │ └── DbSeeder.cs// Initialer Datenbank-Seed für Administrator (SRP) │ └──
Program.cs// App-Start, DI-Container & Middleware-Pipeline │ └── appsettings.json// Basiskonfiguration
(ConnectionStrings, JWT, Smtplib) │ └── Dockerfile// Multi-Stage Docker-Build für .NET 8 Runtime └──
frontend/ │ └── app// Next.js 14 App Router │ │ └── layout.tsx// Root Layout mit NextAuth

```

SessionProvider | | |— page.tsx// Adressenübersicht & Schnellfilter | | |— login/ | | | |—
 page.tsx// Anmeldeformular | | |— adressen/ | | | |— neu/ | | | |— page.tsx// Adresse neu
 erfassen | | | |— [id]/bearbeiten/ | | | |— page.tsx// Adresse bearbeiten (dynamische Route) | |
 |— api/auth/[...nextauth]/ | | |— route.ts// NextAuth API Route Handler | |— components/ | |
 |— AdresseForm.tsx// Wiederverwendbares Formular mit PLZ-Autocomplete | | |—
 ConfirmDialog.tsx// Barrierefreier modaler Bestätigungsdialog | |— lib/ | | |— api.ts// Zentrale fetch-
 Hilfe mit automatischer Bearer-JWT-Übergabe | | |— auth.ts// NextAuth Credentials Provider & Token-
 Callbacks | |— types/ | | |— adresse.ts// TypeScript Interface für Adresse & Audit | | |— city.ts//
 TypeScript Interface für City | | |— next-auth.d.ts// Module Augmentation für Custom Session Claims |
 |— middleware.ts// Edge Route-Protection für geschützte Pfade | |— package.json// Node.js
 Abhängigkeiten & Skripte | |— Dockerfile// Standalone Next.js Container Build |— documentation//
 Zentrale Projektdokumentation | |— Tutorial_WebApp_v2.0.pdf// Früheres Handbuch Version 2.0 |
 |— Tutorial_WebApp_v2.1.pdf// Aktuelles Handbuch Version 2.1 (dieses Dokument) | |—
 Aenderungsprotokoll_v2.1.0.pdf// Protokoll der Clean-Code-Refactorings | |—
 Aenderungsprotokoll_v2.2.0.pdf// Protokoll der JWT- und Struktur-Erweiterungen | |—
 CleanCode_Analyse.pdf// Ursprünglicher Befundbericht mit 14 Befunden |— scripts/// Wartungs- und
 Automatisierungsskripte | |— import_cities.py// Python-Batch-Import der Schweizer PLZ-Daten | |—
 create_user.sh// Shell-Skript zur manuellen Benutzererstellung in DB | |— reset_password.sh// Shell-
 Skript für Notfall-Passwort-Resets |— archive/// Versionierte Release-Pakete zur Nachverfolgung | |—
 AdressverwaltungApp_v2.0.0.zip// Stand v2.0.0 (Basisversion vor Refactoring) | |—
 AdressverwaltungApp_v2.1.0.zip// Stand v2.1.0 (Clean-Code-Refactoring abgeschlossen) | |—
 AdressverwaltungApp_v2.2.0.zip// Stand v2.2.0 (JWT-Integration & Ordnerstruktur) |— docker-
 compose.yml// Multi-Container Orchestrierung (db, backend, frontend)

2.2 Warum diese Struktur?

In Softwareprojekten entscheidet die Verzeichnisstruktur massgeblich darüber, wie wartbar eine Anwendung über Jahre hinweg bleibt. Wir folgen hier bewährten Architekturprinzipien:

1. Strikte Schichtentrennung (backend/ vs. frontend/)

Das C#-Backend und das Next.js-Frontend besitzen völlig unterschiedliche Laufzeitumgebungen, Build-Werkzeuge, Paketmanager (`dotnet` / `NuGet` vs. `npm` / `package.json`) und Lebenszyklen. Durch die Trennung auf oberster Ebene können Frontend- und Backend-Entwickler unabhängig voneinander arbeiten, ohne dass Versionskonflikte in Konfigurationsdateien entstehen. Zudem erlaubt dies eine differenzierte Rechtevergabe und separate CI/CD-Pipelines.

2. Zentrales Dokumentationsverzeichnis (documentation/)

Technische Dokumentation gerät in der Praxis oft ins Hintertreffen oder wird verstreut abgelegt. Indem alle Handbücher, Änderungsprotokolle und Architektur-Reviews versioniert im Ordner `documentation/` direkt neben dem Code liegen, bleiben Dokumentation und Code stets synchron («Docs as Code»). Neue Teammitglieder finden alle relevanten Spezifikationen und Beschreibungen sofort an einem einzigen Ort.

3. Ausgelagerte Automatisierungsskripte (scripts/)

Datenbank-Initialisierung, Datenimporte und administrative Notfallskripte (wie das Zurücksetzen eines vergessenen Kennworts) gehören weder in die Business-Logik des Backends noch in das Frontend. Im Verzeichnis `scripts/` sind diese Werkzeuge isoliert abgelegt. Sie können von DevOps-Ingenieuren oder Administratoren auf der Konsole ausgeführt werden, ohne die Produktiv-Binaries zu belasten.

4. Versioniertes Release-Archiv (archive/)

Im Ordner `archive/` werden produktionsreife Quellcode-Zips vorangegangener Meilensteine reproduzierbar aufbewahrt. Sollte es bei einem Refactoring zu unvorhergesehenen Seiteneffekten kommen, kann der exakte Code-Stand früherer Versionen (v2.0.0, v2.1.0) blitzschnell zur forensischen Analyse herangezogen oder auf Testsystemen deployed werden.

Best Practice für Git-Repositories

In Produktivumgebungen werden temporäre Build-Artefakte wie `bin/`, `obj/`, `node_modules/` und `.next/` über die `.gitignore`-Datei konsequent von der Versionskontrolle ausgeschlossen. Sie lassen sich jederzeit über `dotnet build` bzw. `npm install` deterministisch reproduzieren.

3. Entwicklungsumgebung einrichten

Lernziele dieses Kapitels

- Die lokale Entwicklungsumgebung mit allen erforderlichen SDKs und Tools einrichten
- Das Multi-Container-Setup über Docker Compose fehlerfrei hochfahren und überwachen
- Die Zuordnung von Host-Ports und Container-Ports im Entwicklungsnetzwerk verstehen
- Umgebungsvariablen im Frontend und Backend konfigurieren (inklusive Doppelunterstrich-Konvention)
- Sich mit dem initialen Standard-Administrator anmelden und das System testen

3.1 Voraussetzungen & Werkzeuge

Um die Adressverwaltungs-WebApp lokal zu entwickeln, zu debuggen und auszuführen, werden folgende Softwarekomponenten auf dem Entwicklungsrechner benötigt:

Tabelle 3.1 Erforderliche Softwarekomponenten

Komponente	Mindestversion	Zweck	Prüfbefehl im Terminal
Docker Desktop	24.0+ (Compose v2)	Containerisierung aller Dienste (PostgreSQL, Backend, Frontend)	<code>docker compose version</code>
.NET 8 SDK	8.0.100 LTS	Kompilierung, Paketverwaltung und lokales Ausführen des C#-Backends	<code>dotnet --version</code>
Node.js & npm	Node 20.10+ (LTS)	JavaScript/TypeScript-Laufzeitumgebung für Next.js und npm	<code>node -v && npm -v</code>
Git	2.40+	Quellcode-Verwaltung und Branching	<code>git --version</code>
Visual Studio Code	Aktuell	Integrierte Entwicklungsumgebung (IDE) für Full-Stack	<code>code --version</code>

Empfohlene VS Code Erweiterungen

Für ein produktives Arbeiten in der hybriden C#- und TypeScript-Welt empfehlen wir die Installation folgender Erweiterungen aus dem Visual Studio Marketplace:

- **C# Dev Kit (Microsoft):** Offizielle Erweiterung für C#, Projektverwaltung, Intellisense und Debugging mit Roslyn-Engine.
- **Tailwind CSS IntelliSense:** Autovervollständigung für Tailwind-Klassennamen, Hover-Vorschau von CSS-Regeln und Syntax-Highlighting.
- **ESLint & Prettier:** Automatisierte Code-Formatierung und statische Fehleranalyse für TypeScript/React.
- **Docker (Microsoft):** Visuelle Verwaltung laufender Container, Images, Netzwerke und Volumes direkt in VS Code.
- **Database Client / PostgreSQL:** Leichter Zugriff auf die PostgreSQL-Tabellen ohne externes GUI-Tool.

3.2 Docker Compose starten

Dank Docker Compose lässt sich das gesamte System mit allen Abhängigkeiten ohne aufwendige manuelle Datenbankinstallation mit einem einzigen Terminalbefehl hochfahren.

Terminal – Im Projektstamm ausführen

```
# In das Projektverzeichnis wechseln
cd adressverwaltung/

# Alle Container bauen und im Hintergrund (detached) starten
docker compose up -d --build

# Status der Container überprüfen
docker compose ps
```

Beim ersten Aufruf lädt Docker das PostgreSQL-Image herunter, baut das C#-Backend über das Multi-Stage `Dockerfile` und kompiliert die Next.js-Frontend-Anwendung. Nach Abschluss zeigt `docker compose ps` drei aktive Dienste mit dem Status `Up`:

Ausgabe von `docker compose ps`

NAME		IMAGE	COMMAND
SERVICE	STATUS	PORTS	
adressverwaltung-db		postgres:16-alpine	"docker-entrypoint.s..."
db	Up (healthy)	0.0.0.0:5432->5432/tcp	
adressverwaltung-backend		adressverwaltung-backend:latest	"dotnet Adressverwa..."
backend	Up	0.0.0.0:5000->8080/tcp	
adressverwaltung-frontend		adressverwaltung-frontend:latest	"docker-entrypoint.s..."
frontend	Up	0.0.0.0:3000->3000/tcp	

3.3 Dienste, Ports und URLs

Das Gesamtsystem besteht aus drei Hauptkomponenten, die über ein gemeinsames internes Docker-Bridge-Netzwerk kommunizieren und definierte Ports an den Entwicklungsrechner (Host) weiterleiten:

Tabelle 3.2 Übersicht der Dienste, Netzwerk-Ports und URLs

Dienst	Interner Port	Host-Port	URL / Zugriff	Beschreibung
Frontend	3000	3000	http://localhost:3000	Next.js Web-Portal (Login, Adressenliste, Formulare)
Backend API	8080	5000	http://localhost:5000	C# ASP.NET Core REST & OData Schnittstelle
Swagger UI	8080	5000	http://localhost:5000/swagger	Interaktive API-Dokumentation (nur in Development aktiv)
PostgreSQL	5432	5432	localhost:5432	PostgreSQL 16 Datenbank (User: postgres, DB: adressverwaltung)

3.4 Umgebungsvariablen & Konfiguration

Gemäss den Prinzipien der *Twelve-Factor App* wird Konfiguration strikt vom Code getrennt und über Umgebungsvariablen injiziert. Dadurch bleibt derselbe Code in lokalen Entwicklungs-, Staging- und Produktionsumgebungen unverändert lauffähig.

Frontend (.env.local)

Im Ordner `frontend/` steuert die Datei `.env.local` die Verbindung zum Backend und die Absicherung der NextAuth-Session:

```
frontend/.env.local
```

```
# Geheimer Schlüssel zur Verschlüsselung von Sitzungs-Cookies (min. 32 Zeichen)
NEXTAUTH_SECRET=ein-sehr-sicherer-geheimer-nextauth-schlüssel-mindestens-32-zeichen!

# Öffentliche Basis-URL des Frontends
NEXTAUTH_URL=http://localhost:3000

# Öffentliche URL des Backends für clientseitige Aufrufe im Browser
NEXT_PUBLIC_API_URL=http://localhost:5000

# Interne Docker-URL für serverseitige Next.js Aufrufe (Node.js zu ASP.NET Core)
INTERNAL_API_URL=http://backend:8080
```

Backend (appsettings.json)

Das C#-Backend liest Standardkonfigurationen aus `appsettings.json` :

backend/AdressverwaltungApi/appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=adressverwaltung;Username=postgres;Password=deinPasswort123"
  },
  "Jwt": {
    "Key": "AENDERN_dein-geheimer-schlüssel-mindestens-32-zeichen-fuer-hmac-sha256!",
    "Issuer": "AdressverwaltungApi",
    "Audience": "AdressverwaltungFrontend",
    "ExpiresInHours": 8
  },
  "SmtpSettings": {
    "Host": "smtp.example.com",
    "Port": "587",
    "Username": "benutzer@example.com",
    "Password": "geheimesPasswort",
    "FromAddress": "noreply@example.com",
    "SenderName": "Adressverwaltung System"
  },
  "Seed": {
    "AdminEmail": "admin@example.com",
    "AdminPassword": "Test-1234!"
  }
}
```

Wichtig: Die Docker Doppelunterstrich-Regel (Hierarchie-Mapping)

In Linux- und Docker-Umgebungen sind Doppelpunkte `:` in Umgebungsvariablenamen unzulässig. ASP.NET Core besitzt daher einen standardisierten Konfigurations-Provider: **Zwei aufeinanderfolgende Unterstriche `__` werden automatisch in einen Doppelpunkt übersetzt.**

- `Jwt__Key` überschreibt `Jwt:Key` aus der `appsettings.json`
- `ConnectionStrings__DefaultConnection` überschreibt `ConnectionStrings:DefaultConnection`
- `SmtpSettings__FromAddress` überschreibt `SmtpSettings:FromAddress`

Prüfen Sie bei Konfigurationsfehlern immer zuerst, ob in der `docker-compose.yml` exakt zwei Unterstriche verwendet wurden. Ein einzelner Unterstrich wird nicht als Sektions-Trennzeichen erkannt!

3.5 Standard-Login & Erstanmeldung

Um nach dem ersten Start sofort arbeitsfähig zu sein, ohne zuvor manuell SQL-Statements ausführen zu müssen, besitzt das Backend die Klasse `DbSeeder.cs`.

Beim Hochfahren prüft der Seeder, ob bereits ein Datensatz in der Tabelle `Users` existiert. Ist die Tabelle leer, wird automatisch ein vorkonfigurierter System-Administrator angelegt:

Standard-Zugangsdaten für die Erstanmeldung

```
URL:      http://localhost:3000/login
E-Mail:   admin@example.com
Passwort: Test-1234!
Rolle:    Administrator
```

Nach erfolgreicher Anmeldung wird das JWT im Sitzungs-Cookie hinterlegt und der Benutzer gelangt zur Adressübersicht. Das Kennwort kann anschliessend in der Datenbank oder über das Skript `reset_password.sh` geändert werden.

4. Datenbankmodell

Lernziele dieses Kapitels

- Das relationale Datenbankschema in PostgreSQL und das EF Core O/R-Mapping verstehen
- Die Basisklasse `AuditableEntity` und das automatische Setzen von Audit-Feldern im `DbContext` beherrschen
- Den Unterschied zwischen PostgreSQL `timestamp without time zone` und .NET `DateTimeKind` verstehen
- Die Vor- und Nachteile von `EnsureCreated()` gegenüber klassischen EF-Migrationen in Docker-Umgebungen einordnen
- Das Single Responsibility Principle (SRP) anhand der ausgelagerten `DbSeeder`-Klasse anwenden

4.1 AuditableEntity Basisklasse & Automatischer Audit-Trail

In Unternehmensanwendungen ist die lückenlose Nachvollziehbarkeit von Datensatzänderungen (Compliance & Governance) eine zwingende Anforderung. Niemals sollte ein Datensatz geändert oder gelöscht werden, ohne dass festgehalten wird, **wer** die Operation durchgeführt hat und **wann** dies geschah.

Statt diese Felder redundant in jeder Entität zu deklarieren, erstellen wir eine abstrakte Basisklasse `AuditableEntity` (DRY-Prinzip):

```
backend/AdressverwaltungApi/Models/AuditableEntity.cs
```

```

namespace AdressverwaltungApi.Models;

/// <summary>
/// Abstrakte Basisklasse für alle auditierbaren Entitäten.
/// Beinhaltet Erstellungs- und Änderungsstempel sowie einen fachlichen
/// Gültigkeitszeitraum.
/// </summary>
public abstract class AuditableEntity
{
    public int Id { get; set; }

    // Technischer Audit-Trail
    public DateTime CreateDate { get; set; }
    public string CreatedBy { get; set; } = string.Empty;
    public DateTime? ChangeDate { get; set; }
    public string? ChangedBy { get; set; }

    // Fachlicher Gültigkeitszeitraum (Temporal Data)
    public DateOnly DateFrom { get; set; }
    public DateOnly? DateTo { get; set; }
}

```

Automatisches Befüllen im AdresseDbContext (SaveChanges Override)

Entwickler sollten nicht manuell in jedem Controller `adresse.CreateDate = DateTime.Now` tippen müssen. Solche manuellen Zuweisungen werden schnell vergessen und führen zu inkonsistenten Daten. Stattdessen überschreiben wir `SaveChanges` und `SaveChangesAsync` im `AdresseDbContext` :

```
Auszug aus AdresseDbContext.cs – Automatischer Audit-Trail
```

```

public override int SaveChanges()
{
    ApplyAuditFields();
    return base.SaveChanges();
}

public override Task<int> SaveChangesAsync(CancellationToken cancellationToken =
default)
{
    ApplyAuditFields();
    return base.SaveChangesAsync(cancellationToken);
}

private void ApplyAuditFields()
{
    // Aktuellen Benutzer aus dem HTTP-Kontext ermitteln (JWT Claims)
    var currentUser = _httpContextAccessor.HttpContext?.User?.Identity?.Name

    _httpContextAccessor.HttpContext?.User?.FindFirst("email")?.Value
    ?? "System";

    // WICHTIG für PostgreSQL: DateTimeKind.Unspecified verwenden!
    var now = DateTime.SpecifyKind(DateTime.UtcNow, DateTimeKind.Unspecified);
    var today = DateOnly.FromDateTime(DateTime.UtcNow);

    foreach (var entry in ChangeTracker.Entries<AuditableEntity>())
    {
        if (entry.State == EntityState.Added)
        {
            entry.Entity.CreateDate = now;
            entry.Entity.CreatedBy = currentUser;
            if (entry.Entity.DateFrom == default)
            {
                entry.Entity.DateFrom = today;
            }
        }
        else if (entry.State == EntityState.Modified)
        {
            entry.Entity.ChangeDate = now;
            entry.Entity.ChangedBy = currentUser;
        }
    }
}

```

Kritischer PostgreSQL-Fallstrick: DateTimeKind.Unspecified

PostgreSQL unterscheidet streng zwischen `timestamp without time zone` und `timestamp with time zone`. Wenn eine .NET-Eigenschaft vom Typ `DateTime` den `Kind = DateTimeKind.Utc` besitzt, wirft der Npgsql-Treiber bei Spalten vom Typ `timestamp without time zone` folgenden Laufzeitfehler:

```
Cannot write DateTime with Kind=UTC to PostgreSQL type 'timestamp without time zone'
```

Die Lösung: Wandeln Sie den Zeitstempel mit `DateTime.SpecifyKind(DateTime.UtcNow, DateTimeKind.Unspecified)` um, bevor er an Entity Framework Core übergeben wird!

4.2 Entitäten-Übersicht

Das System verwaltet vier Hauptentitäten, die in PostgreSQL als eigenständige Tabellen abgebildet werden:

Tabelle 4.1 Datenbanktabellen und Entitätsmodell

Entität / Klasse	Tabelle	Wichtigste Eigenschaften / Spalten	Zweck
Adresse	Adressen	Vorname, Name, Strasse, Strassennummer, Plz, Ort (+ Audit-Felder)	Personen- und Firmenadressen mit vollständiger Anschrift
City	Cities	PostalCode (max 10), CityName (max 100) (+ Audit-Felder)	Stammdaten Schweizer Ortschaften für Autocomplete
User	Users	Email (Unique), DisplayName, PasswordHash (PBKDF2)	Benutzerkonten für Login und Berechtigungsprüfung
Settings	Settings	NotificationEmail, EnableEmailNotifications	Globale Systemkonfiguration (Singleton-Tabelle)

C#-Modelle mit DataAnnotations (B-07)

Gemäss Clean-Code-Befund B-07 werden Validierungsregeln direkt über DataAnnotations an den Modellen definiert:

```
backend/AdressverwaltungApi/Models/Adresse.cs
```

```

using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Models;

public class Adresse : AuditableEntity
{
    [Required(ErrorMessage = "Vorname ist ein Pflichtfeld.")]
    [MaxLength(100)]
    public string Vorname { get; set; } = string.Empty;

    [Required(ErrorMessage = "Name ist ein Pflichtfeld.")]
    [MaxLength(100)]
    public string Name { get; set; } = string.Empty;

    [Required(ErrorMessage = "Strasse ist ein Pflichtfeld.")]
    [MaxLength(200)]
    public string Strasse { get; set; } = string.Empty;

    [MaxLength(20)]
    public string Strassennummer { get; set; } = string.Empty;

    [Required(ErrorMessage = "PLZ ist ein Pflichtfeld.")]
    [MaxLength(10)]
    public string Plz { get; set; } = string.Empty;

    [Required(ErrorMessage = "Ort ist ein Pflichtfeld.")]
    [MaxLength(100)]
    public string Ort { get; set; } = string.Empty;
}

```

backend/AdressverwaltungApi/Models/City.cs

```

using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Models;

public class City : AuditableEntity
{
    [Required]
    [MaxLength(10)]
    public string PostalCode { get; set; } = string.Empty;

    [Required]
    [MaxLength(100)]
    public string CityName { get; set; } = string.Empty;
}

```

backend/AdressverwaltungApi/Models/User.cs & Settings.cs

```

using System.ComponentModel.DataAnnotations;

namespace AdressverwaltungApi.Models;

public class User
{
    public int Id { get; set; }

    [Required, EmailAddress, MaxLength(256)]
    public string Email { get; set; } = string.Empty;

    [Required, MaxLength(100)]
    public string DisplayName { get; set; } = string.Empty;

    // NIEMALS im Klartext speichern!
    [Required]
    public string PasswordHash { get; set; } = string.Empty;
}

public class Settings
{
    public int Id { get; set; }

    [Required, EmailAddress, MaxLength(256)]
    public string NotificationEmail { get; set; } = string.Empty;

    public bool EnableEmailNotifications { get; set; } = true;
}

```

4.3 EnsureCreated statt Migrations

In Enterprise-Produktionssystemen werden Datenbankschemas üblicherweise über relationale Migrationsskripte (`dotnet ef migrations add`) versioniert. In lokalen Schulungs- und Docker-Umgebungen führt dies jedoch regelmässig zu schwer verständlichen Fehlern:

- Wird ein Docker-Volume (`pgdata`) nicht gelöscht, enthält die Datenbank alte Tabellenstände, während der C#-Code bereits neuere Migrationen erwartet.
- Es kommt zum gefürchteten Fehler: `relation "Users" does not exist` oder Migrations-Konflikten in der Tabelle `__EFMigrationsHistory`.

Für diese WebApp setzen wir deshalb beim Start auf `EnsureCreated()` :

```
Program.cs - Schema-Initialisierung
```

```
// EnsureCreated() prüft, ob die Datenbank existiert.
// Falls nicht, erstellt es alle Tabellen, Primärschlüssel und Indizes
// direkt und atomar aus dem aktuellen Entity Framework Core Modell.
db.Database.EnsureCreated();
```

Tabelle 4.2 Vergleich: EnsureCreated() vs. EF Core Migrations

Kriterium	EnsureCreated()	EF Core Migrations (dotnet ef database update)
Funktionsweise	Erstellt Tabellen direkt aus C#-Klassen	Wendet inkrementelle C#-Migrationsdateien an
Tracking-Tabelle	Keine <code>__EFMigrationsHistory</code> nötig	Erfordert <code>__EFMigrationsHistory</code> synchrone
Docker-Robustheit	Sehr hoch – startet immer zuverlässig	Mittel – anfällig für Volume-Zustände
Schema-Updates im Betrieb	Kann bestehende Tabellen nicht alterieren	Unterstützt <code>ALTER TABLE</code> ohne Datenverlust
Einsatzgebiet	Entwicklung, CI/CD-Tests, Lehrmittel	Produktivbetrieb mit bestehenden Datenbeständen

4.4 DbSeeder (Single Responsibility – B-01)

Im Befund B-01 der Clean-Code-Analyse wurde kritisiert, dass der Seed-Code zum Anlegen des Administrators direkt in der `Program.cs` platziert war. Dies verletzte das **Single Responsibility Principle (SRP)**: Die Startup-Datei soll ausschliesslich Middleware und Dependency Injection konfigurieren.

Die Initialisierungslogik wurde daher in eine eigenständige Klasse `DbSeeder.cs` ausgelagert. Zusätzlich werden E-Mail und Kennwort nicht mehr hartkodiert («Magic Strings»), sondern aus der Konfiguration bezogen:

```
backend/AdressverwaltungApi/DbSeeder.cs
```

```

using Microsoft.AspNetCore.Identity;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;

namespace AdressverwaltungApi;

/// <summary>
/// Befüllt die Datenbank beim ersten Anwendungsstart mit initialen Stammdaten.
/// Erfüllt das Single Responsibility Principle (SRP).
/// </summary>
public static class DbSeeder
{
    public static void Seed(
        AdresseDbContext db,
        IPasswordHasher<User> hasher,
        IConfiguration config)
    {
        // 1. Admin-Benutzer anlegen (nur wenn Users leer ist)
        if (!db.Users.Any())
        {
            var adminEmail = config["Seed:AdminEmail"] ?? "admin@example.com";
            var adminPassword = config["Seed:AdminPassword"] ?? "Test-1234!";

            var admin = new User
            {
                Email = adminEmail.Trim().ToLowerInvariant(),
                DisplayName = "Administrator"
            };

            // Kennwort sicher mit PBKDF2 und Salt hashen
            admin.PasswordHash = hasher.HashPassword(admin, adminPassword);

            db.Users.Add(admin);
        }

        // 2. Standard-Einstellungen anlegen (nur wenn Settings leer ist)
        if (!db.Settings.Any())
        {
            db.Settings.Add(new Settings
            {
                NotificationEmail = config["Seed:AdminEmail"] ?? "admin@example.com",
                EnableEmailNotifications = true
            });
        }

        db.SaveChanges();
    }
}

```

5. Backend – ASP.NET Core 8

Lernziele dieses Kapitels

- Die Struktur der `Program.cs` und die Konfiguration des Dependency Injection Containers in .NET 8 beherrschen
- Die kritische Bedeutung der Middleware-Reihenfolge (Pipeline) für Authentifizierung und Autorisierung verstehen
- Das OData v4 Protokoll für flexible, standardisierte REST-Datenabfragen anwenden
- Einen generischen CRUD-Controller (`ODataCrudController<TEntity>`) zur Einhaltung des DRY-Prinzips implementieren

5.1 Program.cs – Übersicht der registrierten Dienste

In .NET 8 fungiert `Program.cs` als zentraler Einstiegspunkt der Web API. Hier werden Dienste im **Inversion of Control (IoC) Container** registriert und die HTTP-Verarbeitungspipeline (Middleware) zusammengestellt.

```
backend/AdressverwaltungApi/Program.cs (Vollständige Konfiguration)
```

```

using Microsoft.AspNetCore.OData;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OData.ModelBuilder;
using Microsoft.EntityFrameworkCore;
using System.Text;
using AdressverwaltungApi;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;
using AdressverwaltungApi.Services;

var builder = WebApplication.CreateBuilder(args);

// 1. DATENBANK: PostgreSQL via Entity Framework Core
builder.Services.AddDbContext<AdresseDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection"))
);

// 2. HTTP-KONTEXT: Für Audit-Felder (CreatedBy/ChangedBy aus JWT Claims)
builder.Services.AddHttpContextAccessor();

// 3. PASSWORT-HASHING: ASP.NET Core PasswordHasher (PBKDF2 mit 10'000+ Iterationen & Salt)
builder.Services.AddScoped<IPasswordHasher<User>, PasswordHasher<User>>();

// 4. DIENSTE: E-Mail & Fachliche Benachrichtigung (Interfaces für Austauschbarkeit)
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<INotificationService, EmailNotificationService>();

// 5. JWT-AUTHENTIFIZIERUNG: Token-Validierung konfigurieren
var jwtKey = builder.Configuration["Jwt:Key"]
    ?? throw new InvalidOperationException("Jwt:Key fehlt in der Konfiguration!");
var keyBytes = Encoding.UTF8.GetBytes(jwtKey);

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(keyBytes),
        ValidateIssuer = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"] ??
            "AdressverwaltungApi",
        ValidateAudience = true,
        ValidAudience = builder.Configuration["Jwt:Audience"] ??
            "AdressverwaltungFrontend",
    }
});

```

```

        ValidateLifetime          = true,
        ClockSkew                  = TimeSpan.Zero // Keine Verzögerungstoleranz beim
Ablauf
    };
});

builder.Services.AddAuthorization();

// 6. ODATA: Entity Data Model (EDM) definieren
var modelBuilder = new ODataConventionModelBuilder();
modelBuilder.EnableLowerCamelCase();
modelBuilder.EntitySet<Adresse>("Adressen");
modelBuilder.EntitySet<City>("Cities");

// 7. CONTROLLER & ODATA QUERY OPTIONS
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.PropertyNamingPolicy =
            System.Text.Json.JsonNamingPolicy.CamelCase;
    })
    .AddOData(options => options
        .Select()
        .Filter()
        .OrderBy()
        .SetMaxTop(100)
        .Count()
        .Expand()
        .AddRouteComponents("odata", modelBuilder.GetEdmModel())
    );

// 8. CORS: Zugriff vom Frontend erlauben
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins(
            "http://localhost:3000",
            "http://frontend:3000"
        )
        .AllowAnyHeader()
        .AllowAnyMethod()
        .AllowCredentials();
    });
});

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// 9. DATENBANK INITIALISIERUNG BEIM START
using (var scope = app.Services.CreateScope())
{

```

```

var db      = scope.ServiceProvider.GetRequiredService<AdresseDbContext>();
var hasher = scope.ServiceProvider.GetRequiredService<IPasswordHasher<User>>();
var config = scope.ServiceProvider.GetRequiredService<IConfiguration>();

db.Database.EnsureCreated();
DbSeeder.Seed(db, hasher, config);
}

// 10. MIDDLEWARE-PIPELINE (REIHENFOLGE BEACHTEN!)
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowFrontend");
app.UseRouting();

app.UseAuthentication(); // 1. Wer bist du? (Liest JWT aus Header)
app.UseAuthorization();  // 2. Was darfst du? (Prüft Rollen & [Authorize])

app.MapControllers();

app.Run();

```

5.2 Middleware-Reihenfolge – Ein fataler Anfängerfehler

In ASP.NET Core werden eingehende HTTP-Requests sequenziell durch eine Kette von Komponenten geleitet – die sogenannte **Middleware-Pipeline**. Jede Komponente kann den Request untersuchen, manipulieren oder direkt beantworten:

HTTP Request | ▼ [UseCors] — Prüft Origin, setzt Access-Control-Allow-* Header | ▼ [UseRouting] — Findet den passenden Controller-Endpunkt | ▼ [UseAuthentication] — Liest "Authorization: Bearer <JWT>", validiert Signatur, | befüllt HttpContext.User mit Claims ▼ [UseAuthorization] — Prüft [Authorize] Attribute am Controller gegen HttpContext.User | ▼ [MapControllers] — Führt die Controller-Action-Methode aus

Warnung: UseAuthentication MUSS vor UseAuthorization stehen!

Einer der häufigsten und frustrierendsten Fehler in Web APIs ist das Vertauschen dieser beiden Zeilen:

Häufiger Fehler

```
// FALSCH – Führt immer zu 401 Unauthorized!
app.UseAuthorization();
app.UseAuthentication();
```

Warum? Wenn `UseAuthorization()` zuerst aufgerufen wird, wurde der JWT-Token noch gar nicht ausgewertet! `HttpContext.User.Identity.IsAuthenticated` ist noch `false`. Der Autorisierungsscheck schlägt sofort fehl und sendet ein **HTTP 401 Unauthorized** an den Client – obwohl der Client einen perfekten, gültigen Token mitgesendet hat!

5.3 OData – Was ist das?

Klassische REST-Schnittstellen erfordern für jede Filtersituation oft eigene Endpunkte (z. B. `/api/adressen/searchByCity?name=Bern`, `/api/adressen/sortedByName`). Dies führt zu einer Explosion von Controller-Methoden.

OData (Open Data Protocol) ist ein von OASIS standardisiertes Protokoll, das mächtige Abfrage-Operatoren direkt über URL-Parameter ermöglicht. Der Clou: Entity Framework Core übersetzt diese URL-Parameter direkt in performante SQL-Queries auf der Datenbank!

Tabelle 5.1 Wichtigste OData-Abfrageoperatoren

OData-Operator	Bedeutung	Beispiel-URL
<code>\$filter</code>	Filtiert Zeilen nach Kriterien	<code>/odata/Adressen?\$filter=Ort eq 'Zürich'</code>
<code>\$select</code>	Projiziert spezifische Spalten	<code>/odata/Adressen?\$select=Vorname, Name, Ort</code>
<code>\$orderby</code>	Sortiert das Resultat (asc / desc)	<code>/odata/Adressen?\$orderby=Name asc, Vorname desc</code>
<code>\$top</code>	Begrenzt die Anzahl Treffer (Paging)	<code>/odata/Cities?\$top=10</code>
<code>\$skip</code>	Überspringt Datensätze (Paging)	<code>/odata/Adressen?\$skip=20&\$top=10</code> (Seite 3)
<code>\$count</code>	Gibt Gesamtzahl der Treffer zurück	<code>/odata/Adressen?\$count=true</code>
<code>startswith()</code>	Prüft Präfix (Ideal für Autocomplete)	<code>/odata/Cities?\$filter=startswith(postalCode, '80')</code>

OData-Beispielabfragen in der Praxis

Typische Anwendungsfälle im Projekt

```
# 1. Alle Adressen in Bern, sortiert nach Nachname:
GET /odata/Adressen?$filter=Ort eq 'Bern'&$orderby=Name

# 2. PLZ-Autocomplete: Die ersten 10 Schweizer Orte, die mit '80' beginnen:
GET /odata/Cities?$filter=startswith(postalCode, '80')&$top=10&$orderby=postalCode

# 3. Adressensuche mit Vor- und Nachname kombiniert:
GET /odata/Adressen?$filter=contains(tolower(Name), 'meier') or
contains(tolower(Vorname), 'meier')
```

5.4 Generischer ODataCrudController (DRY – B-02, B-08)

In v2.0 besaßen `AdressenController` und `CitiesController` nahezu identischen Code für GET, POST, PATCH und DELETE. Gemäss Clean-Code-Befund B-08 wurde diese Duplikation durch die Einführung einer abstrakten generischen Basisklasse `ODataCrudController<TEntity>` beseitigt:

```
backend/AdressverwaltungApi/Controllers/ODataCrudController.cs
```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.OData.Routing.Controllers;
using Microsoft.AspNetCore.OData.Query;
using Microsoft.AspNetCore.OData.Deltas;
using Microsoft.AspNetCore.Authorization;
using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi.Data;

namespace AdressverwaltungApi.Controllers;

/// <summary>
/// Generische Basisklasse für OData-CRUD-Controller.
/// DRY-Prinzip: Vollständige CRUD-Logik wird nur einmal implementiert.
/// [Authorize] schützt automatisch alle abgeleiteten Controller.
/// </summary>
[Authorize]
public abstract class ODataCrudController

```

```

        Entities.Add(entity);
        await _context.SaveChangesAsync();

        return Created(entity);
    }

    // PATCH /odata/{EntitySet}(5)
    public virtual async Task<IActionResult> Patch(
        [FromRoute] int key,
        [FromBody] Delta<TEntity> delta)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var entity = await Entities.FindAsync(key);
        if (entity is null)
            return NotFound($"{EntityDisplayName} mit Id {key} wurde nicht gefunden.");

        delta.Patch(entity);

        if (!TryValidateModel(entity))
            return BadRequest(ModelState);

        await _context.SaveChangesAsync();

        return Updated(entity);
    }

    // DELETE /odata/{EntitySet}(5)
    public virtual async Task<IActionResult> Delete([FromRoute] int key)
    {
        var entity = await Entities.FindAsync(key);
        if (entity is null)
            return NotFound($"{EntityDisplayName} mit Id {key} wurde nicht gefunden.");

        Entities.Remove(entity);
        await _context.SaveChangesAsync();

        return NoContent();
    }
}

```

Konkrete Implementierungen: Elegant und schlank

Dank der generischen Basisklasse schrumpft der `CitiesController` auf wenige Zeilen zusammen:

```
backend/AdressverwaltungApi/Controllers/CitiesController.cs
```

```

using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;

namespace AdressverwaltungApi.Controllers;

public class CitiesController : ODataCrudController<City>
{
    protected override DbSet<City> Entities => _context.Cities;
    protected override string EntityDisplayName => "Stadt";

    public CitiesController(AdresseDbContext context) : base(context)
    {
    }
}

```

Und der `AdressenController` muss lediglich die `Post`-Methode überschreiben, um nach der Persistierung die E-Mail-Benachrichtigung über das `INotificationService`-Interface auszulösen:

```

backend/AdressverwaltungApi/Controllers/AdressenController.cs

```

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using AdressverwaltungApi.Data;
using AdressverwaltungApi.Models;
using AdressverwaltungApi.Services;

namespace AdressverwaltungApi.Controllers;

public class AdressenController : ODataCrudController<Adresse>
{
    protected override DbSet<Adresse> Entities => _context.Adressen;
    protected override string EntityDisplayName => "Adresse";

    private readonly INotificationService _notificationService;

    public AdressenController(
        AdresseDbContext context,
        INotificationService notificationService) : base(context)
    {
        _notificationService = notificationService;
    }

    public override async Task<IActionResult> Post([FromBody] Adresse adresse)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        Entities.Add(adresse);
        await _context.SaveChangesAsync();

        // Asynchron benachrichtigen (SRP: Entkoppelt via Service)
        await _notificationService.NotifyNewAdresseAsync(adresse);

        return Created(adresse);
    }
}

```

6. JWT-Authentifizierung

NEU in v2.1

Lernziele dieses Kapitels

- Den Aufbau eines JSON Web Tokens (Header, Payload, Signatur) nach RFC 7519 verstehen
- Die Vor- und Nachteile von zustandslosen JWTs gegenüber serverseitigen Sessions abwägen
- Die JWT-Bearer-Authentifizierung in ASP.NET Core 8 von Grund auf konfigurieren
- Sichere Token-Generierung im `AuthController` mit benutzerdefinierten Claims implementieren
- Das NextAuth-Typensystem in TypeScript via Module Augmentation erweitern
- Den vollständigen Authentifizierungsfluss von der Formulareingabe bis zum geschützten API-Aufruf nachvollziehen

6.1 Was ist ein JWT?

Ein **JSON Web Token (JWT)** ist ein offener Industrie-Standard (RFC 7519), der ein kompaktes, URL-sicheres Format für den vertrauenswürdigen Austausch von Informationen zwischen zwei Kommunikationspartnern definiert.

Ein JWT besteht aus drei durch Punkte (.) getrennten Zeichenketten, die jeweils Base64URL-kodiert sind:

Header
(Kopfdaten)

Payload (Nutzlast /
Claims)

Signature (Kryptografische
Signatur)

1. Header (Kopfdaten)

Der Header definiert den Typ des Tokens sowie das verwendete Signatur- bzw. Verschlüsselungsverfahren:

Header (JSON dekodiert)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

HS256 steht für *HMAC mit SHA-256*, ein symmetrisches Signaturverfahren, bei dem Absender und Empfänger denselben geheimen Schlüssel teilen.

2. Payload (Claims)

Der Payload enthält die eigentlichen Aussagen (*Claims*) über den authentifizierten Benutzer sowie Metadaten zum Token. RFC 7519 unterscheidet zwischen reservierten, öffentlichen und privaten Claims:

Payload (JSON dekodiert)

```
{
  "sub": "42",
  "email": "admin@example.com",
  "name": "Administrator",
  "jti": "d3b07384-d113-4f4c-b9b3-4f51e03a98f1",
  "exp": 1727452800,
  "iss": "AdressverwaltungApi",
  "aud": "AdressverwaltungFrontend"
}
```

Tabelle 6.1 Wichtige Standard-Claims im Payload

Claim	Bezeichnung	Bedeutung
sub	Subject	Die eindeutige Identifikationsnummer des Benutzers (z. B. User-ID 42).
email	E-Mail	Die E-Mail-Adresse des authentifizierten Benutzers.
name	Display Name	Der Anzeigename für Begrüssungen in der Benutzeroberfläche.
jti	JWT ID	Einzigartige GUID des Tokens zur Verhinderung von Replay-Angriffen.
exp	Expiration Time	Ablaufzeitpunkt des Tokens als Unix-Timestamp (Sekunden seit 01.01.1970).
iss	Issuer	Die ausstellende Instanz (hier: <code>AdressverwaltungApi</code>).
aud	Audience	Der vorgesehene Empfänger des Tokens (hier: <code>AdressverwaltungFrontend</code>).

3. Signature (Kryptografische Signatur)

Die Signatur wird berechnet, indem der Base64URL-kodierte Header und der Base64URL-kodierte Payload mit einem geheimen Schlüssel über den HMAC-SHA256-Algorithmus gehasht werden:

Signaturberechnung

```
HMACSHA256(
    base64UrlEncode(header) + "." + base64UrlEncode(payload),
    geheimer_schlüssel
)
```

Praxistipp: Token inspizieren mit jwt.io

Auf der Website <https://jwt.io> befindet sich ein interaktiver Debugger. Kopieren Sie ein beliebiges JWT in das Eingabefeld: Header und Payload werden sofort in lesbares JSON dekodiert. **Merke:** Der Payload ist lediglich Base64-kodiert, *nicht verschlüsselt*! Jeder, der das Token besitzt, kann die Claims lesen. Speichern Sie daher **niemals** Passwörter, Kreditkartennummern oder vertrauliche Firmengeheimnisse im Payload.

6.2 Warum JWT statt serverseitiger Sessions?

Klassische Web-Architekturen verwenden oft zustandsbehaftete Sessions (Stateful Sessions): Nach dem Login legt der Server ein Session-Objekt im Arbeitsspeicher oder in einer Redis-Datenbank ab und sendet eine zufällige Session-ID als Cookie an den Browser.

In modernen Microservices- und entkoppelten Architekturen stösst dieses Modell an Grenzen:

Tabelle 6.2 Vergleich: JWT (Zustandslos) vs. Server-Sessions (Zustandsbehaftet)

Kriterium	JWT (Stateless Token)	Server-Sessions (Stateful Cookie)
Speicherort des Zustands	Vollständig im Token auf Clientseite	Im RAM oder in zentraler Session-DB des Servers
Skalierbarkeit (Scale-Out)	Beliebig viele Backend-Instanzen können Token unabhängig validieren	Erfordert Sticky Sessions oder geteilten Cache (Redis, Memcached)
Datenbanklast	Null DB-Abfragen pro Request zur Auth-Prüfung	Eine DB- oder Cache-Abfrage bei jedem einzelnen HTTP-Request
Cross-Domain & Mobile	Funktioniert nahtlos über Domains, Mobile Apps und APIs hinweg	Cookies sind an Domains gebunden und erfordern komplexe CORS-Regeln
Token-Invalidierung	Token bleibt bis zum <code>exp</code> -Ablauf gültig (oder erfordert Blacklist)	Server kann Sessions sofort per Knopfdruck serverseitig löschen
Payload-Grösse	Grösserer HTTP-Header (~500-1000 Bytes)	Sehr kleiner Cookie (~32 Bytes)

6.3 JWT im Backend konfigurieren

Die Implementierung der JWT-Validierung in ASP.NET Core 8 erfolgt in drei präzisen Schritten:

Schritt 1: NuGet-Paket einbinden

In der Projektdatei `backend/AdressverwaltungApi/AdressverwaltungApi.csproj` wird das offizielle Microsoft-Paket hinzugefügt:

AdressverwaltungApi.csproj (Auszug)

```
<ItemGroup>
  <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer"
    Version="8.0.0" />
</ItemGroup>
```

Schritt 2: Parameter in appsettings.json definieren

appsettings.json

```
"Jwt": {
  "Key": "dein-sehr-geheimer-schlüssel-mindestens-32-zeichen-lang!",
  "Issuer": "AdressverwaltungApi",
  "Audience": "AdressverwaltungFrontend",
  "ExpiresInHours": 8
}
```

Schritt 3: Authentifizierungsdienst in Program.cs registrieren

Program.cs (Auszug)

```

var jwtKey = builder.Configuration["Jwt:Key"]
    ?? throw new InvalidOperationException("Jwt:Key ist nicht konfiguriert!");

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            // 1. Signatur mit unserem geheimen Schlüssel prüfen
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),

            // 2. Aussteller (Issuer) prüfen
            ValidateIssuer = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"] ??
                "AdressverwaltungApi",

            // 3. Empfänger (Audience) prüfen
            ValidateAudience = true,
            ValidAudience = builder.Configuration["Jwt:Audience"] ??
                "AdressverwaltungFrontend",

            // 4. Ablaufzeit prüfen
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero // Keine Verzögerungstoleranz
        };
    });

builder.Services.AddAuthorization();

```

6.4 JWT ausstellen – Der AuthController

Wenn ein Benutzer seine Anmeldedaten an `POST /auth/login` sendet, überprüft der `AuthController` das Kennwort. Ist die Prüfung erfolgreich, wird die private Hilfsmethode `GenerateJwt(User user)` aufgerufen:

```

backend/AdressverwaltungApi/Controllers/AuthController.cs (Auszug)

```

```

private string GenerateJwt(User user)
{
    var jwtKey = _config["Jwt:Key"]
        ?? throw new InvalidOperationException("Jwt:Key fehlt.");
    var keyBytes = Encoding.UTF8.GetBytes(jwtKey);
    var securityKey = new SymmetricSecurityKey(keyBytes);
    var credentials = new SigningCredentials(securityKey,
        SecurityAlgorithms.HmacSha256);

    var hours = double.Parse(_config["Jwt:ExpiresInHours"] ?? "8");

    // Benutzerdefinierte Claims zusammenstellen
    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
        new Claim(JwtRegisteredClaimNames.Email, user.Email),
        new Claim(JwtRegisteredClaimNames.Name, user.DisplayName),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    var token = new JwtSecurityToken(
        issuer: _config["Jwt:Issuer"] ?? "AdressverwaltungApi",
        audience: _config["Jwt:Audience"] ?? "AdressverwaltungFrontend",
        claims: claims,
        expires: DateTime.UtcNow.AddHours(hours),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Die Login Action-Methode mit LoginResponse DTO

Gemäss Clean Code verwenden wir für die Antwort ein stark typisiertes C# Record DTO `LoginResponse` (Befund B-02 / B-04):

backend/AdressverwaltungApi/Dtos/AuthDtos.cs

```

namespace AdressverwaltungApi.Dtos;

public record LoginRequest(string Email, string Password);

public record LoginResponse(
    string Id,
    string Name,
    string Email,
    string Token
);

```

```
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequest req)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    // E-Mail case-insensitiv nachschlagen (B-06: ToUpperInvariant für Culture-Safety)
    var emailNorm = req.Email.Trim().ToUpperInvariant();
    var user = await _context.Users.FirstOrDefaultAsync(u => u.Email.ToUpper() ==
emailNorm);

    // Timing-Angriff-Prävention: Immer Verification durchführen
    var dummyUser = new User();
    var hashToVerify = user?.PasswordHash ?? _hasher.HashPassword(dummyUser, "dummy");
    var verifyTarget = user ?? dummyUser;
    var result = _hasher.VerifyHashedPassword(verifyTarget, hashToVerify,
req.Password);

    if (user is null || result == PasswordVerificationResult.Failed)
    {
        // Generische Fehlermeldung (kein User-Enumeration-Leak)
        return Unauthorized(new { message = "Ungültige Anmeldedaten." });
    }

    // JWT generieren und im LoginResponse zurückgeben
    var tokenString = GenerateJwt(user);

    return Ok(new LoginResponse(
        Id: user.Id.ToString(),
        Name: user.DisplayName,
        Email: user.Email,
        Token: tokenString
    ));
}
```

6.5 Endpunkte schützen mit [Authorize]

Das Sicherheitskonzept unterscheidet streng zwischen öffentlichen und geschützten Ressourcen:

- **Automatisch geschützt:** Alle von `ApiController<TEntity>` abgeleiteten Controller (`AdressenController`, `CitiesController`) sowie der `SettingsController` tragen das Attribut `[Authorize]`. Anfragen ohne gültigen Bearer-Token werden von ASP.NET Core sofort mit `401 Unauthorized` abgewiesen.

- **Öffentlich zugänglich:** Der `AuthController` besitzt **kein** `[Authorize]`-Attribut. Die Endpunkte `/auth/login` und `/auth/register` müssen für anonyme Benutzer erreichbar sein, damit diese sich überhaupt erst authentifizieren können.

6.6 JWT im Frontend speichern (NextAuth.js)

Im Next.js Frontend verwaltet **NextAuth.js** die Benutzersitzung. Standardmässig speichert NextAuth nur Basisdaten wie Name und E-Mail. Wir erweitern NextAuth so, dass das vom Backend ausgestellte JWT im verschlüsselten Sitzungs-Cookie mitgeführt wird.

TypeScript Module Augmentation (F-04)

Um Typsicherheit ohne unsichere Typ-Casts (`as any`) zu gewährleisten, erweitern wir die NextAuth-Typdefinitionen in `frontend/types/next-auth.d.ts` :

frontend/types/next-auth.d.ts

```
import 'next-auth';
import { DefaultSession } from 'next-auth';

declare module 'next-auth' {
  interface User {
    accessToken?: string; // Unser C#-Backend-JWT
  }

  interface Session {
    accessToken?: string; // Steht Client-Komponenten via getSession() zur Verfügung
    user: {
      id: string;
    } & DefaultSession['user'];
  }
}

declare module 'next-auth/jwt' {
  interface JWT {
    id?: string;
    accessToken?: string; // Gespeichert im HTTP-Only Session Cookie
  }
}
```

NextAuth-Konfiguration in frontend/lib/auth.ts

In `auth.ts` konfigurieren wir den `CredentialsProvider` und verknüpfen die Callbacks `jwt` und `session` :

frontend/lib/auth.ts (Auszug)

```

import CredentialsProvider from 'next-auth/providers/credentials';
import type { NextAuthOptions } from 'next-auth';

export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      name: 'Credentials',
      credentials: {
        email: { label: 'E-Mail', type: 'email' },
        password: { label: 'Passwort', type: 'password' }
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) return null;

        const apiUrl = process.env.INTERNAL_API_URL || 'http://backend:8080';
        const res = await fetch(`${apiUrl}/auth/login`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            email: credentials.email,
            password: credentials.password
          })
        });

        if (!res.ok) return null;

        const data = await res.json();
        // data entspricht dem LoginResponse Record aus dem Backend:
        // { id, name, email, token }
        return {
          id: data.id,
          name: data.name,
          email: data.email,
          accessToken: data.token // JWT an NextAuth übergeben
        };
      }
    })
  ],
  callbacks: {
    // 1. JWT-Callback: Token aus authorize() in den verschlüsselten Cookie schreiben
    async jwt({ token, user }) {
      if (user) {
        token.id = user.id;
        token.accessToken = user.accessToken;
      }
      return token;
    },

    // 2. Session-Callback: Token aus Cookie für React-Komponenten freigeben
    async session({ session, token }) {
      if (session.user) {
        session.user.id = token.id as string;
      }
    }
  }
};

```

```
        session.accessToken = token.accessToken;
        return session;
    }
},
pages: {
    signIn: '/login'
},
session: {
    strategy: 'jwt'
}
};
```

6.7 API-Aufrufe mit JWT (frontend/lib/api.ts)

Damit Frontend-Komponenten nicht bei jedem Aufruf manuell Header zusammenstellen müssen, kapselt eine zentrale Hilfsfunktion `apiFetch` das Auslesen der Session und das Anhängen des `Authorization: Bearer <JWT>` -Headers:

```
frontend/lib/api.ts (Auszug)
```

```

import { getSession } from 'next-auth/react';

const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:5000';

/// <summary>
/// Führt einen authentifizierten fetch()-Aufruf an das C#-Backend aus.
/// Hängt automatisch den Bearer-Token der aktuellen Sitzung an.
/// </summary>
export async function apiFetch(endpoint: string, options: RequestInit = {}):
Promise<Response> {
  const session = await getSession();
  const token = session?.accessToken;

  const headers: Record<string, string> = {
    'Content-Type': 'application/json',
    ...(options.headers as Record<string, string> ?? {})
  };

  // JWT als Bearer-Token anhängen, falls vorhanden
  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const url = endpoint.startsWith('http') ? endpoint : `${API_BASE_URL}${endpoint}`;
  return fetch(url, {
    ...options,
    headers
  });
}

// Beispiel: Alle Adressen abrufen
export async function getAdressen(): Promise<Adresse[]> {
  const res = await apiFetch('/odata/Adressen');
  if (!res.ok) {
    throw new Error(`Fehler beim Laden der Adressen (HTTP ${res.status})`);
  }
  const json = await res.json();
  return json.value.map(normalizeAdresse);
}

```

6.8 Vollständiger Login-Ablauf Schritt für Schritt

Das Zusammenspiel aller beteiligten Schichten lässt sich in einer 11-stufigen Kette nachvollziehen:

- 1 **Benutzer gibt Zugangsdaten ein:** Auf der Seite `/login` tippt der Benutzer `admin@example.com` und sein Kennwort ein.

- 2 **NextAuth signIn():** Das Formular ruft `signIn('credentials', { email, password })` im Browser auf.
- 3 **Server-to-Server Aufruf:** NextAuth führt die Methode `authorize()` aus und sendet `POST /auth/login` an das C#-Backend.
- 4 **Passwort-Validierung im Backend:** Der `AuthController` liest den `PasswordHash` aus PostgreSQL und validiert das Passwort über `IPasswordHasher`.
- 5 **JWT generieren:** Bei Erfolg signiert `GenerateJwt()` das Token mit dem geheimen HMAC-SHA256 Schlüssel und gibt `LoginResponse` zurück.
- 6 **Cookie-Verschlüsselung:** NextAuth empfängt das JWT im `jwt`-Callback und verschlüsselt es sicher mit `NEXTAUTH_SECRET` in einem HTTP-Only Cookie.
- 7 **Weiterleitung:** Der Browser wird zur Hauptseite `/` weitergeleitet. Die Adressenliste lädt.
- 8 **apiFetch() liest Token:** Die Frontend-Komponente ruft `getAdressen()` auf. `apiFetch()` extrahiert das Backend-JWT aus der Session.
- 9 **HTTP-Request mit Bearer:** Ein Request an `GET /odata/Adressen` wird gesendet mit dem Header `Authorization: Bearer eyJhbGciOi...`
- 10 **Kryptografische Prüfung:** Die ASP.NET Core JWT-Middleware prüft die Signatur und den Ablaufzeitpunkt `exp`. `HttpContext.User` wird mit Claims befüllt.
- 11 **Datenlieferung:** Der `AdressenController` liefert den Adressbestand als OData-JSON-Array aus. Das Frontend rendert die Tabelle.

6.9 Sicherheitshinweise für die Praxis

Checkliste für den Produktivbetrieb

- **Schlüssellänge:** Der HMAC-SHA256 Algorithmus verlangt einen Schlüssel von **mindestens 256 Bit (32 Byte / 32 Zeichen)**. Kürzere Schlüssel führen zu Laufzeitfehlern oder lassen sich leicht per Brute-Force brechen.

- **Keine Geheimnisse in Git:** Verwenden Sie im Produktivbetrieb niemals den Standardschlüssel aus der `appsettings.json`. Setzen Sie den Schlüssel über die Umgebungsvariable `Jwt__Key` auf dem Server.
- **Angemessene Gültigkeit:** Die Ablaufzeit ist auf **8 Stunden** konfiguriert (typische Arbeitsschicht). Für Hochsicherheitsanwendungen empfiehlt sich ein kürzeres JWT (z. B. 15 Minuten) in Kombination mit Refresh-Tokens.
- **Keine sensiblen Daten im Payload:** Da der Payload öffentlich lesbar ist, dürfen dort keine Passwörter, Hashes oder privaten Stammdaten hinterlegt werden.
- **HTTPS erzwingen:** Da der Bearer-Token bei jedem Aufruf im Klartext-Header übertragen wird, muss der Datenverkehr zwingend über TLS/HTTPS verschlüsselt werden.

7. Frontend – Next.js 14

Lernziele dieses Kapitels

- Die moderne Next.js 14 App-Router-Architektur mit Layouts, Pages und dynamischen Routen verstehen
- Globalen Routenschutz auf dem Edge-Server über NextAuth `middleware.ts` konfigurieren
- Eine performante Autocomplete-Wertehilfe mit React `useEffect`, Debouncing und OData umsetzen
- React Best Practices anwenden: Durchgehend Controlled Inputs, `useCallback`-Optimierung und barrierefreie Modaldialoge

7.1 App-Router-Struktur in Next.js 14

Next.js 14 setzt standardmässig auf den **App Router** (Verzeichnis `frontend/app/`). Im Gegensatz zum früheren *Pages Router* basiert das Routing rein auf der Verzeichnisstruktur und nutzt **React Server Components (RSC)** als Standard:

frontend/app/ |— layout.tsx// Root Layout: SessionProvider, Navigation & globales Styling |—
 page.tsx// Startseite: Adressenliste, Filter & Schnellsuche |— loading.tsx// Automatischer Lade-Skeleton
 während Server-Side Rendering |— error.tsx// Error Boundary zur Abfangung unvorhergesehener Fehler
 |— login/ | |— page.tsx// Anmeldeseite mit E-Mail- und Passwort-Formular |— adressen/ |— neu/
 | |— page.tsx// Formular zur Neuerfassung einer Adresse |— [id]/bearbeiten/ |— page.tsx//
 Dynamische Route: Bearbeitung der Adresse mit gegebener ID

Root Layout mit SessionProvider (frontend/app/layout.tsx)

Damit Client-Komponenten über den Hook `useSession()` auf Benutzerdaten und das JWT zugreifen können, muss die Applikation in einen `SessionProvider` gehüllt werden:

```
frontend/app/layout.tsx
```

```

import type { Metadata } from 'next';
import './globals.css';
import Providers from '@components/Providers';

export const metadata: Metadata = {
  title: 'Adressverwaltung v2.1',
  description: 'Enterprise Adressverwaltung mit Next.js 14 & C# OData',
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="de">
      <body className="bg-slate-50 text-slate-900 min-h-screen">
        <Providers>
          <nav className="bg-blue-900 text-white shadow-md px-6 py-4 flex justify-between items-center">
            <div className="font-bold text-lg">Adressverwaltung v2.1</div>
          </nav>
          <main className="max-w-6xl mx-auto p-6">
            {children}
          </main>
        </Providers>
      </body>
    </html>
  );
}

```

7.2 Routen-Schutz (frontend/middleware.ts)

Anstatt auf jeder Einzelseite manuell zu prüfen, ob der Benutzer eingeloggt ist, nutzen wir die Edge Middleware von Next.js. Sie fängt Requests bereits ab, bevor eine Seite überhaupt gerendert wird:

```
frontend/middleware.ts
```

```
import { withAuth } from 'next-auth/middleware';

export default withAuth({
  pages: {
    signIn: '/login', // Nicht angemeldete Besucher hierhin umleiten
  },
});

export const config = {
  // Schützt alle Pfade ausser Login, API-Auth und statischen Assets
  matcher: [
    '/((?!login|api/auth|_next/static|_next/image|favicon.ico).*)',
  ],
};
```

7.3 PLZ-Autocomplete mit Debouncing

Das Erfassen von Schweizer Postleitzahlen soll für den Benutzer schnell und fehlerfrei ablaufen. Tippt der Anwender beispielsweise `80` ein, schlägt die Komponente automatisch «8001 Zürich», «8002 Zürich» usw. vor.

Warum Debouncing unverzichtbar ist

Würde bei jedem einzelnen Tastenanschlag sofort eine API-Abfrage an das Backend gesendet, entstünde bei schnellem Tippen eine Flut von HTTP-Requests. Dies belastet das Netzwerk und die Datenbank unnötig. Ein **Debounce-Timer** wartet ab, bis der Benutzer für **300 Millisekunden** keine Taste mehr gedrückt hat, und feuert erst dann den Request:

Auszug aus AdresseForm.tsx – Debounced Autocomplete

```

const [plz, setPlz] = useState(initialAdresse?.plz || '');
const [ort, setOrt] = useState(initialAdresse?.ort || '');
const [vorschlaege, setVorschlaege] = useState<City[]>([]);
const [sucheLaeuft, setSucheLaeuft] = useState(false); // F-05: Korrigierte Namensgebung

useEffect(() => {
  // Erst ab 2 Ziffern suchen
  if (plz.trim().length < 2) {
    setVorschlaege([]);
    return;
  }

  setSucheLaeuft(true);
  const timer = setTimeout(async () => {
    try {
      // OData startswith-Abfrage ausführen
      const query = `/odata/Cities?
$filter=startswith(postalCode, '${encodeURIComponent(plz.trim())}')&$top=8&$orderby=post
alCode`;
      const res = await apiFetch(query);
      if (res.ok) {
        const data = await res.json();
        setVorschlaege(data.value || []);
      }
    } catch (err) {
      console.error('Fehler bei der PLZ-Suche:', err);
    } finally {
      setSucheLaeuft(false);
    }
  }, 300); // 300 ms Beruhigungszeit

  // Cleanup-Funktion: Löscht den vorherigen Timer bei erneutem Tastendruck
  return () => clearTimeout(timer);
}, [plz]);

```

7.4 React Best Practices

1. Durchgehend Controlled Inputs (Befund F-02)

Im Befund F-02 der Clean-Code-Analyse wurde kritisiert, dass `AdresseForm.tsx` teilweise uncontrolled (`defaultValue`) und teilweise controlled (`value` / `onChange`) implementiert war. Diese Mischform führt zu unvorhersehbarem Synchronisationsverhalten.

In v2.1 sind **ausnahmslos alle Formularfelder als Controlled Inputs** via React State angebunden:

Konsistente Controlled Inputs

```



```

2. Performance-Optimierung mit useCallback (Befund F-06)

In React wird eine Funktion, die innerhalb einer Komponente deklariert ist, bei jedem Render-Durchlauf neu instanziiert. Wird diese Funktion als Property an untergeordnete Komponenten übergeben, führt das zu unnötigen Re-Renders.

Mit `useCallback` erhält die Funktion eine stabile Referenz:

frontend/app/page.tsx - useCallback

```

const ladeDaten = useCallback(async () => {
  setLaden(true);
  try {
    const daten = await getAdressen();
    setAdressen(daten);
  } catch (err) {
    console.error(err);
  } finally {
    setLaden(false);
  }
}, []); // Leeres Dependency-Array = Funktion bleibt über Lebenszyklus stabil

useEffect(() => {
  ladeDaten();
}, [ladeDaten]);

```

3. Barrierefreier ConfirmDialog statt window.confirm (Befund F-03)

Die browser-eigenen Dialoge `alert()` und `confirm()` sind Relikte der frühen Web-Ära: Sie blockieren den gesamten UI-Thread, können vom Betriebssystem unterdrückt werden und lassen sich weder stylen noch barrierefrei bedienen.

In v2.1 wird eine eigenständige, modal gerenderte React-Komponente `ConfirmDialog.tsx` eingesetzt:

frontend/components/ConfirmDialog.tsx

```

interface ConfirmDialogProps {
  isOpen: boolean;
  title: string;
  message: string;
  onConfirm: () => void;
  onCancel: () => void;
}

export default function ConfirmDialog({
  isOpen,
  title,
  message,
  onConfirm,
  onCancel,
}: ConfirmDialogProps) {
  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center bg-black/50
backdrop-blur-sm">
      <div className="bg-white rounded-lg shadow-xl max-w-md w-full p-6 animate-in
fade-in">
        <h3 className="text-lg font-bold text-slate-900 mb-2">{title}</h3>
        <p className="text-slate-600 mb-6">{message}</p>
        <div className="flex justify-end gap-3">
          <button
            onClick={onCancel}
            className="px-4 py-2 border rounded text-slate-700 hover:bg-slate-100"
          >
            Abbrechen
          </button>
          <button
            onClick={onConfirm}
            className="px-4 py-2 bg-red-600 text-white rounded hover:bg-red-700 font-
medium"
          >
            Löschen bestätigen
          </button>
        </div>
      </div>
    </div>
  );
}

```

8. Clean Code – Umgesetzte Massnahmen (v2.1.0)

Lernziele dieses Kapitels

- Die 14 Befunde der Clean-Code-Analyse und deren systematische Behebung nachvollziehen
- Das Single Responsibility Principle (SRP) und das DRY-Prinzip in C# und TypeScript in der Praxis anwenden
- Verstehen, wie Code-Refactoring die Wartbarkeit, Robustheit und Testbarkeit drastisch erhöht
- Statische Typprüfung und Culture-Safety zur Vermeidung tückischer Produktionsfehler einsetzen

Im Rahmen des Release-Zyklus v2.1.0 wurde der gesamte Quellcode der Adressverwaltung einer umfassenden Code-Review nach den Grundsätzen von Robert C. Martin («Clean Code») und den SOLID-Entwurfsprinzipien unterzogen. Dabei wurden **14 konkrete Schwachstellen** identifiziert und vollständig behoben.

8.1 Backend-Befunde (B-01 bis B-08)

Tabelle 8.1 Behobene Backend-Schwachstellen (C# .NET 8)

ID	Prinzip	Betroffene Datei	Ursprüngliches Problem (Vorher)	Umgesetzte Lösung (Nachher)
B-01	SRP	<code>Program.cs</code>	Seed-Code (Admin-User anlegen) mit Magic Strings direkt in Startup-Datei.	In eigene Klasse <code>DbSeeder.cs</code> ausgelagert. Zugangsdaten via <code>IConfiguration</code> .
B-02	SRP / Struktur	<code>AuthController.cs</code> , <code>SettingsController.cs</code>	DTOs (<code>LoginRequest</code> , <code>RegisterRequest</code>) im Controller deklariert.	In separate Dateien im neuen Ordner <code>Dtos/AuthDtos.cs</code> und <code>Dtos/SettingsDtos.cs</code> extrahiert.
B-03	SRP / OCP	<code>AdressenController.cs</code>	E-Mail-Zusammenbau und Versandlogik direkt in Controller-Methode.	Interface <code>INotificationService</code> und Service <code>EmailNotificationService.cs</code> erstellt.

B-04	Robustheit	EmailService.cs	int.Parse()	wirft unbehandelte FormatException bei fehlerhafter Konfiguration.	Auf sicheres int.TryParse() mit Standard-Fallback 587 umgestellt.
B-05	Konfiguration	EmailService.cs docker-compose.yml	/ Inkonsistenter Schlüssel: Code las SenderEmail, Docker setzte FromAddress.	Einheitlich auf SmtplibSettings.FromAddress und SmtplibSettings__FromAddress harmonisiert.	
B-06	Culture-Safety	AuthController.cs	E-Mail-Vergleich mit .ToLower() ist von System-Locale abhängig (türkisches İ-Problem).	Auf .ToUpperInvariant() umgestellt (EF Core SQL-konform und culture-safe).	
B-07	Validierung	Models/Adresse.cs	Keine Validierungs-Attribute am Model (Längenbeschränkungen fehlten).	[Required] und [MaxLength] DataAnnotations direkt an den Properties deklariert.	
B-08	DRY	AdressenController.cs, CitiesController.cs	Identische CRUD-Methoden (GET/POST/PATCH/DELETE) in beiden Controllern dupliziert.	Gemeinsame Basisklasse ODataCrudController<TEntity> eingeführt.	

Detail-Refactoring: B-06 Culture-Safe String-Vergleich

Ein klassischer Fehler in Web-Backends ist der unbedachte Aufruf von `string.ToLower()`. In bestimmten Systemumgebungen (z. B. türkische Sprach-Locale `tr-TR`) wird der Grossbuchstabe `I` zu einem punktlosen kleinen `ı` konvertiert – und **nicht** zu `i`! Ein Benutzer namens `info@firma.ch` könnte sich auf einem türkisch konfigurierten Linux-Server niemals anmelden.

Vorher vs. Nachher – Culture-Safe E-Mail-Vergleich

```
// ✗ VORHER: Locale-abhängig und fehleranfällig
var exists = await _context.Users.AnyAsync(u => u.Email.ToLower() == req.Email.ToLower());

// ✓ NACHHER: Invariante Grossschreibung (Culture-Safe & EF Core Index-kompatibel)
var emailNormalized = req.Email.Trim().ToUpperInvariant();
var exists = await _context.Users.AnyAsync(u => u.Email.ToUpper() == emailNormalized);
```

8.2 Frontend-Befunde (F-01 bis F-06)

Tabelle 8.2 Behobene Frontend-Schwachstellen (TypeScript & React)

ID	Prinzip	Betroffene Datei	Ursprüngliches Problem (Vorher)	Umgesetzte Lösung (Nachher)
F-01	DRY	<code>frontend/lib/api.ts</code>	Audit-Feld-Normalisierung (6 Felder) in <code>normalizeAdresse</code> und <code>normalizeCity</code> dupliziert.	Hilfsfunktion <code>normalizeAuditFields()</code> extrahiert und in beiden Normalizern wiederverwendet.
F-02	UI-Konsistenz	<code>components/AdresseForm.tsx</code>	Mischung aus Controlled (PLZ/Ort) und Uncontrolled (Vorname, Name via <code>defaultValue</code>).	Alle Eingabefelder einheitlich als Controlled Inputs via <code>useState</code> implementiert.
F-03	UX / A11y	<code>app/page.tsx</code>	Verwendung von blockierendem <code>window.confirm()</code> und <code>alert()</code> beim Löschen.	Eigene barrierefreie Modal-Komponente <code>ConfirmDialog.tsx</code> mit State integriert.
F-04	Typsicherheit	<code>lib/auth.ts</code>	Unsicherer Type-Cast <code>(session.user as { id?: string }).id</code> zur Umgehung von TypeScript.	Saubere NextAuth Module Augmentation in <code>types/next-auth.d.ts</code> deklariert.
F-05	Konvention	<code>components/AdresseForm.tsx</code>	Tippfehler in State-Variable: <code>suchelaeuft</code> statt camelCase <code>sucheLaeuft</code> .	Auf <code>const [sucheLaeuft, setSucheLaeuft] = useState(false);</code> korrigiert.
F-06	Performance	<code>app/page.tsx</code>	<code>ladeDaten</code> war eine unmemoized Funktion; erzeugte bei jedem Render neue Referenzen.	Mit <code>useCallback(async () => { ... }, [])</code> stabilisiert.

Detail-Refactoring: F-01 DRY Audit-Feld-Normalisierung

Da OData je nach Serialisierungseinstellungen Schlüssel in Klein- oder Grossschreibung liefern kann, müssen die 6 Audit-Felder (`createDate`, `createdBy`, `changeDate`, `changedBy`, `dateFrom`, `dateTo`) normalisiert werden. Früher war dieser 15-zeilige Block in jeder Mapper-Funktion kopiert:

frontend/lib/api.ts – Extrahierte Hilfsfunktion

```
//  Einmal zentral definiert:
function normalizeAuditFields(raw: Record<string, unknown>) {
  return {
    createDate: String(raw['createDate'] ?? raw['CreateDate'] ?? ''),
    createdBy: String(raw['createdBy'] ?? raw['CreatedBy'] ?? ''),
    changeDate: (raw['changeDate'] ?? raw['ChangeDate'] ?? null) as string | null,
    changedBy: (raw['changedBy'] ?? raw['ChangedBy'] ?? null) as string | null,
    dateFrom: String(raw['dateFrom'] ?? raw['DateFrom'] ?? ''),
    dateTo: (raw['dateTo'] ?? raw['DateTo'] ?? null) as string | null,
  };
}

// Schlanker Einsatz in beiden Entitäten:
function normalizeAdresse(raw: Record<string, unknown>): Adresse {
  return {
    id: Number(raw['id'] ?? raw['Id']),
    vorname: String(raw['vorname'] ?? raw['Vorname'] ?? ''),
    name: String(raw['name'] ?? raw['Name'] ?? ''),
    strasse: String(raw['strasse'] ?? raw['Strasse'] ?? ''),
    strassennummer: String(raw['strassennummer'] ?? raw['Strassennummer'] ?? ''),
    plz: String(raw['plz'] ?? raw['Plz'] ?? ''),
    ort: String(raw['ort'] ?? raw['Ort'] ?? ''),
    ...normalizeAuditFields(raw), // DRY!
  };
}
```

9. Docker Compose

Lernziele dieses Kapitels

- Die Multi-Container-Orchestrierung mit Docker Compose verstehen und anpassen können
- Das Konzept von `healthcheck` und `depends_on: condition: service_healthy` zur Steuerung der Startreihenfolge beherrschen
- Den Unterschied zwischen benannten Volumes (Data Persistence) und Bind Mounts kennen
- Die wichtigsten CLI-Befehle zur Diagnose, Log-Analyse und Container-Verwaltung sicher anwenden

9.1 Dienste-Konfiguration (docker-compose.yml)

Im Wurzelverzeichnis des Projekts definiert die Datei `docker-compose.yml` die gesamte Entwicklungs-Infrastruktur. Sie verknüpft die drei Services (Datenbank, Backend und Frontend) in einem isolierten virtuellen Docker-Bridge-Netzwerk:

```
docker-compose.yml (Vollständige Konfiguration)
```

```

services:
# -----
# 1. DATENBANK: PostgreSQL 16 Relationale Datenbank
# -----
db:
  image: postgres:16-alpine
  container_name: adressverwaltung-db
  restart: unless-stopped
  environment:
    POSTGRES_DB:      adressverwaltung
    POSTGRES_USER:    postgres
    POSTGRES_PASSWORD: deinPasswort123
  ports:
    - "5432:5432" # Ermöglicht lokalen Zugriff via pgAdmin oder psql
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    # Prüft, ob PostgreSQL tatsächlich Verbindungen entgegennimmt
    test: ["CMD-SHELL", "pg_isready -U postgres -d adressverwaltung"]
    interval: 5s
    timeout: 5s
    retries: 10

# -----
# 2. BACKEND: C# ASP.NET Core 8 Web API
# -----
backend:
  build:
    context: ./backend/AdressverwaltungApi
    dockerfile: Dockerfile
  container_name: adressverwaltung-backend
  restart: unless-stopped
  ports:
    - "5000:8080" # Host 5000 -> Container 8080 (.NET 8 Default)
  environment:
    - ASPNETCORE_ENVIRONMENT=Development

ConnectionStrings__DefaultConnection=Host=db;Port=5432;Database=adressverwaltung;Userna
me=postgres;Password=deinPasswort123
    - Jwt__Key=dein-sehr-geheimer-schluessel-mindestens-32-zeichen-lang!
    - Jwt__Issuer=AdressverwaltungApi
    - Jwt__Audience=AdressverwaltungFrontend
    - Jwt__ExpiresInHours=8
    - SmtptSettings__FromAddress=noreply@example.com
    - SmtptSettings__SenderName=Adressverwaltung System
  depends_on:
    db:
      # Startet das Backend erst, wenn PostgreSQL healthy ist!
      condition: service_healthy

# -----
# 3. FRONTEND: Next.js 14 App Router
# -----

```

```

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: adressverwaltung-frontend
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - NODE_ENV=development
    - NEXTAUTH_URL=http://localhost:3000
    - NEXTAUTH_SECRET=ein-sehr-sicherer-geheimer-nextauth-schluessel-mindestens-32-
zeichen!
    - NEXT_PUBLIC_API_URL=http://localhost:5000
    - INTERNAL_API_URL=http://backend:8080
  depends_on:
    - backend

# Persistente Volumes definieren
volumes:
  postgres_data:
    name: adressverwaltung_postgres_data

```

Warum service_healthy für die Datenbank entscheidend ist

Ein häufiger Irrglaube ist, dass ein einfaches `depends_on: [ db ]` ausreicht. Docker startet den Datenbank-Container zwar zuerst, PostgreSQL benötigt aber oft mehrere Sekunden, um das interne Datenverzeichnis zu initialisieren und den TCP-Port 5432 zu öffnen. Das Backend würde in dieser Zeitspanne sofort abstürzen (`connection refused`).

Durch `healthcheck` mit `pg_isready` wartet Docker Compose exakt so lange, bis der Datenbankserver Verbindungen annimmt. Erst dann wird das Backend gestartet!

9.2 Wichtige Docker-Compose-Befehle

Für die tägliche Arbeit sollten Entwickler folgende Befehle sicher beherrschen:

Tabelle 9.1 Wichtige Docker Compose Befehle für die Entwicklung

Befehl	Beschreibung & typischer Anwendungsfall
<code>docker compose up -d</code>	Startet alle Container im Hintergrund (Detached Mode).
<code>docker compose up -d --build</code>	Baut die Docker-Images neu und startet die Container. Pflicht nach Änderungen am C#- oder Next.js-Code!
<code>docker compose down</code>	Stoppt und entfernt alle Container und Netzwerke. <i>Daten im Volume bleiben erhalten.</i>

<code>docker compose down -v</code>	Stoppt Container und löscht auch alle Volumes (Setzt die PostgreSQL-Datenbank komplett zurück).
<code>docker compose ps</code>	Zeigt den aktuellen Status aller Container, Port-Weiterleitungen und Healthchecks an.
<code>docker compose logs -f backend</code>	Verfolgt die Live-Konsolenausgabe (Logs) des C#-Backends in Echtzeit (Follow-Mode).
<code>docker compose logs -f frontend</code>	Verfolgt die Konsolenausgabe des Next.js-Frontends (z. B. Server Component Fehler).
<code>docker compose restart backend</code>	Startet nur das Backend neu, ohne die Datenbank anzuhalten.
<code>docker compose exec db psql -U postgres -d adressverwaltung</code>	Öffnet eine interaktive PostgreSQL-Kommandozeile (psql) direkt im laufenden Datenbank-Container.

Praxis-Szenario: Datenbank komplett zurücksetzen

Wenn Sie die Datenbank auf den ursprünglichen Zustand zurücksetzen möchten (z. B. um den Seed-Vorgang oder den PLZ-Import erneut zu testen), führen Sie folgende Befehlskette aus:

Terminal – Reset der Entwicklungsdatenbank

```
# 1. Alle Container stoppen und Volumes vernichten (-v)
docker compose down -v

# 2. Neu starten: DbSeeder erstellt Tabellen und Admin automatisch neu
docker compose up -d

# 3. Prüfen, ob die Datenbank gesund ist
docker compose ps
```

10. Hilfsskripte (scripts/)

Lernziele dieses Kapitels

- Die Automatisierung administrativer Aufgaben über das Verzeichnis `scripts/` verstehen
- Das Python-Importskript `import_cities.py` zur Deduplizierung und Übernahme amtlicher Schweizer Geodaten einsetzen
- Verstehen, wie Batch-Inserts mit `psycopg2` und automatische Audit-Felder in Python realisiert werden
- Shell-Skripte für administrative Notfälle (Benutzeranlage, Passwort-Reset) sicher bedienen

10.1 PLZ-Import (scripts/import_cities.py)

Damit die Werthilfe im Frontend funktioniert, muss die Tabelle `Cities` mit validen Daten befüllt werden. Das Bundesamt für Landestopografie (swisstopo) und die Schweizerische Post stellen das *Amtliche Ortschaftsverzeichnis mit Postleitzahl und Perimeter (AMTOVZ)* als offene Behördendaten (Open Government Data, OGD) kostenlos zur Verfügung.

Die Datei `AMTOVZ_CSV_LV95.csv` enthält **5'718 Datenzeilen**. Viele Ortschaften besitzen jedoch Mehrfacheinträge (unterschiedliche Zusatzziffern, Sprachgrenzen oder Gemeindezuteilungen). Ein ungeprüfter Import würde zu massiven Duplikaten in der Autocomplete-Auswahl führen.

Voraussetzungen

Terminal – Abhängigkeiten installieren

```
# Python 3 und PostgreSQL-Treiber installieren
pip install psycopg2-binary
# alternativ unter Ubuntu/Debian:
sudo apt install python3-psycopg2
```

Funktionsweise des Import-Skripts

Das Skript `scripts/import_cities.py` führt einen intelligenten 4-stufigen Import durch:

1. **CSV-Parsing & Deduplizierung:** Die Datei wird mit `utf-8-sig` (zur sicheren Entfernung des UTF-8 Byte Order Marks) und Semikolon-Trennzeichen eingelesen. Ein Python `set()` filtert Duplikate nach dem Schlüsselpaar `(PLZ4, Ortschaftsname)`. Aus 5'718 Zeilen verbleiben exakt **4'073 eindeutige Ortschaften**.
2. **Datenbankverbindung:** Das Skript verbindet sich über `psycopg2` mit der PostgreSQL-Instanz.
3. **Differenzabgleich (Delta-Check):** Vorhandene Datensätze werden aus der Tabelle `Cities` geladen. Es werden **nur noch nicht vorhandene** Ortschaften eingefügt – bestehende Daten werden nicht überschrieben (kein destruktives `TRUNCATE`!).
4. **Performerter Batch-Insert:** Neue Einträge werden in Blöcken von 500 Datensätzen mit automatisch generierten Audit-Zeitstempeln eingefügt.

scripts/import_cities.py – Teil 1: CSV einlesen und deduplizieren

```
#!/usr/bin/env python3
import os, sys, csv, psycopg2

CSV_PATH = os.environ.get("CSV_PATH", "AMTOVZ_CSV_LV95.csv")
DB_HOST = os.environ.get("DB_HOST", "localhost")
DB_PORT = os.environ.get("DB_PORT", "5432")
DB_NAME = os.environ.get("DB_NAME", "adressverwaltung")
DB_USER = os.environ.get("DB_USER", "postgres")
DB_PASS = os.environ.get("DB_PASS", "deinPasswort123")

print(f"[1/4] Lese CSV-Datei: {CSV_PATH}")
unique_cities = {}
with open(CSV_PATH, mode="r", encoding="utf-8-sig") as f:
    reader = csv.DictReader(f, delimiter=";")
    for row in reader:
        plz = row["PLZ4"].strip()
        ort = row["Ortschaftsname"].strip()
        if plz and ort:
            key = (plz, ort)
            if key not in unique_cities:
                unique_cities[key] = (plz, ort)

print(f"          {len(unique_cities)} eindeutige (PLZ, Ortschaftsname)-Kombinationen gefunden.")
```

Im zweiten Schritt verbindet sich das Skript mit der PostgreSQL-Instanz, ermittelt bereits vorhandene Einträge zur Vermeidung von Dubletten und führt den Batch-Insert durch:

scripts/import_cities.py – Teil 2: Datenbankabgleich und Batch-Insert

```

print("[2/4] Verbinde mit Datenbank ...")
conn = psycopg2.connect(
    host=DB_HOST, port=DB_PORT, dbname=DB_NAME, user=DB_USER, password=DB_PASS
)
cur = conn.cursor()
print("    Verbindung erfolgreich.")

print("[3/4] Lade bestehende Einträge aus der Datenbank ...")
cur.execute('SELECT "PostalCode", "CityName" FROM "Cities";')
existing = set(cur.fetchall())
print(f"    {len(existing)} bestehende Einträge in der Tabelle.")

to_insert = [c for c in unique_cities.values() if c not in existing]
print(f"    {len(to_insert)} neue Einträge werden eingefügt.")

print("[4/4] Füge neue Einträge ein ...")
if to_insert:
    insert_sql = '''
        INSERT INTO "Cities"
        ("PostalCode", "CityName", "CreateDate", "CreatedBy", "DateFrom")
        VALUES (%s, %s, NOW() AT TIME ZONE 'UTC', 'ImportScript', CURRENT_DATE);
    '''
    cur.executemany(insert_sql, to_insert)
    conn.commit()
    print(f"    {len(to_insert)} Einträge erfolgreich eingefügt. ✓")
else:
    print("    Keine neuen Einträge erforderlich. Alles aktuell. ✓")

cur.close()
conn.close()
print("\nImport erfolgreich abgeschlossen.")

```

Ausführung im Terminal

Terminal - Import starten

```

# Umgebungsvariablen setzen und Skript ausführen
DB_PASS=deinPasswort123 \
CSV_PATH=./Uploads/AMTOVZ_CSV_LV95.csv \
python3 scripts/import_cities.py

```

Erwartete Konsolenausgabe

```
[1/4] Lese CSV-Datei: ./Uploads/AMTOVZ_CSV_LV95.csv
      4073 eindeutige (PLZ, Ortschaftsname)-Kombinationen gefunden.
[2/4] Verbinde mit Datenbank ...
      Verbindung erfolgreich.
[3/4] Lade bestehende Einträge aus der Datenbank ...
      0 bestehende Einträge in der Tabelle.
      4073 neue Einträge werden eingefügt.
[4/4] Füge neue Einträge ein ...
      4073 Einträge erfolgreich eingefügt. ✓

Import erfolgreich abgeschlossen.
```

10.2 Benutzer anlegen (scripts/create_user.sh)

Möchte ein Administrator einen neuen Benutzer anlegen, ohne dafür das Frontend zu verwenden (oder falls der Admin-Account gesperrt ist), bietet `create_user.sh` eine bequeme Konsolen-Schnittstelle.

Das Skript führt vor dem Insert einen Sicherheitscheck durch: Es prüft über `information_schema.tables`, ob die Tabelle `Users` in PostgreSQL bereits existiert. Anschliessend generiert es einen kompatiblen PBKDF2-Hash und trägt den Benutzer atomar ein:

Terminal - Neuen Benutzer interaktiv anlegen

```
# Skript aufrufen
bash scripts/create_user.sh

# Interaktiver Dialog:
# E-Mail eingeben:   entwickler@example.com
# Name eingeben:    Max Muster
# Passwort eingeben: Geheim-2026!
# [OK] Benutzer entwickler@example.com erfolgreich angelegt.
```

10.3 Passwort zurücksetzen (scripts/reset_password.sh)

Wurde das Kennwort des Administrators vergessen, ermöglicht `reset_password.sh` einen Notfall-Reset direkt auf Datenbankebene:

Terminal - Notfall-Passwort-Reset

```
bash scripts/reset_password.sh

# Das Skript fragt nach der E-Mail des Benutzers und dem neuen Passwort:
# E-Mail: admin@example.com
# Neues Passwort: Neu-Passwort-123!
# [OK] Passwort für admin@example.com erfolgreich aktualisiert.
```

Docker-Integration der Skripte

Alle Skripte im Verzeichnis `scripts/` kommunizieren standardmässig mit `localhost:5432`. Da der PostgreSQL-Port in der `docker-compose.yml` an den Host weitergeleitet ist (`5432:5432`), können die Skripte direkt auf dem Entwicklungsrechner ausgeführt werden, während PostgreSQL im Docker-Container läuft.

11. Häufige Fehler und Lösungen

Lernziele dieses Kapitels

- Typische Fehlerursachen in verteilten Multi-Container-Anwendungen systematisch diagnostizieren
- Verbindungs- und Authentifizierungsfehler zwischen Frontend, Backend und PostgreSQL beheben
- Die Diagnose anhand von Fehlermeldungen (SqlState, HTTP Status Codes) professionalisieren
- Schnelle Troubleshooting-Strategien für den Entwicklungsalltag verinnerlichen

Beim Betrieb und bei der Weiterentwicklung von Full-Stack-Applikationen treten unweigerlich Fehler auf. Dieses Kapitel dient als schnelles Nachschlagewerk für die fünf häufigsten Probleme, ihre genauen Ursachen und die bewährten Lösungswege.

11.1 SqlState: 28P01 – Falsches Datenbankpasswort

Symptom & Fehlermeldung

Das C#-Backend startet nicht und wirft in den Docker-Logs (`docker compose logs backend`) folgende Exception:

```
Npgsql.PostgresException (0x80004005): 28P01: password authentication failed for user "postgres"
```

Ursache: Beim allerersten Start hat PostgreSQL das Volume `postgres_data` mit dem damals gesetzten Passwort initialisiert. Wenn Sie später das Kennwort in der `docker-compose.yml` oder im Connection-String ändern, übernimmt PostgreSQL das neue Passwort *nicht* automatisch, da das bestehende Datenverzeichnis nicht überschrieben wird.

Lösung:

Terminal – Bereinigung des Passwort-Konflikts

```
# 1. Sicherstellen, dass ConnectionStrings__DefaultConnection und POSTGRES_PASSWORD  
identisch sind  
# In docker-compose.yml prüfen:  
# POSTGRES_PASSWORD: deinPasswort123  
# ConnectionStrings__DefaultConnection: ...Password=deinPasswort123  
  
# 2. Bestehendes Volume mit den alten Anmeldedaten vernichten:  
docker compose down -v  
  
# 3. Neu starten (PostgreSQL initialisiert das Verzeichnis mit dem neuen Kennwort):  
docker compose up -d
```

11.2 401 Unauthorized bei API-Aufrufen

Symptom

Der Benutzer ist im Frontend angemeldet, aber beim Aufruf der Adressenliste oder beim Speichern erscheint ein Fehler. In der Browser-Entwicklerkonsole (Netzwerk-Tab) antwortet das Backend mit **HTTP 401 Unauthorized**.

Mögliche Ursachen & Lösungen:

- **Token abgelaufen:** Das JWT besitzt eine konfigurierte Gültigkeitsdauer von 8 Stunden. Nach Ablauf dieser Zeit weist ASP.NET Core das Token automatisch ab.
Lösung: Einmal im Frontend abmelden (Logout) und erneut anmelden.
- **Inkonsistenter JWT-Key:** Wenn das Backend neu gestartet wurde und dabei ein anderer Schlüssel in `Jwt__Key` übergeben wurde als zum Zeitpunkt der Token-Ausstellung, schlägt die Signaturprüfung fehl.
Lösung: Prüfen Sie, ob `Jwt__Key` in der `docker-compose.yml` stabil definiert ist.
- **apiFetch() übergibt keinen Token:** Wurde versehentlich das native `fetch()` anstelle unserer Hilfsfunktion `apiFetch()` verwendet, fehlt der Header `Authorization: Bearer <JWT>`.
Lösung: Immer `apiFetch('/odata/...')` aus `frontend/lib/api.ts` importieren.

11.3 Middleware-Reihenfolge falsch (Program.cs)

Symptom

Obwohl der Bearer-Token im HTTP-Header mitgesendet wird und absolut gültig ist, liefert jeder geschützte Endpunkt ausnahmslos `401 Unauthorized`.

Ursache: In der `Program.cs` des Backends wurde `app.UseAuthorization()` vor `app.UseAuthentication()` aufgerufen.

Lösung:

backend/AdressverwaltungApi/Program.cs

```
// ❌ FALSCH:  
app.UseAuthorization();  
app.UseAuthentication();  
  
// ✅ RICHTIG – Authentifizierung MUSS zuerst erfolgen:  
app.UseAuthentication(); // 1. Liest Token und befüllt User Claims  
app.UseAuthorization(); // 2. Prüft Rollen und Berechtigungen
```

11.4 Leere PLZ-Vorschläge im Adressformular

Symptom

Beim Eintippen einer Schweizer Postleitzahl (z. B. `80`) öffnet sich kein Dropdown-Menü, und die Ortschaft wird nicht vorgeschlagen.

Ursache: Die Tabelle `Cities` in der PostgreSQL-Datenbank ist leer. Der automatische `DbSeeder` legt lediglich den Administrator und die Grundeinstellungen an, importiert jedoch aus Performancegründen nicht die 4'073 Schweizer Ortschaften.

Lösung: Führen Sie das Importskript einmalig manuell aus:

Terminal – Import ausführen

```
DB_PASS=deinPasswort123 \  
CSV_PATH=./Uploads/AMTOVZ_CSV_LV95.csv \  
python3 scripts/import_cities.py
```

11.5 Connection refused :5432 (Verbindung verweigert)

Symptom

Lokale Skripte oder das Backend melden beim Verbindungsaufbau:

```
could not connect to server: Connection refused (0x0000274D / 111) Is the server  
running on host "localhost" and accepting TCP/IP connections on port 5432?
```

Ursache: Der Docker-Daemon läuft nicht, oder der Container `adressverwaltung-db` wurde noch nicht gestartet bzw. ist nach einem Fehler abgestürzt.

Lösung:

Terminal - Diagnose und Start

```
# 1. Prüfen, ob Docker läuft und welche Container aktiv sind:  
docker compose ps  
  
# 2. Falls db nicht "Up", Logs prüfen:  
docker compose logs db  
  
# 3. Container starten:  
docker compose up -d db
```

12. Versionsverlauf

Die Adressverwaltungs-WebApp wurde iterativ in vier Entwicklungsstufen aufgebaut. Die nachfolgende Tabelle dokumentiert den gesamten Evolutionspfad:

Tabelle 12.1 Versionsgeschichte der Adressverwaltungs-WebApp

Version	Datum	Wichtigste Neuerungen & Meilensteine
v1.0.0	Januar 2024	Basis-Prototyp: Einfache Adress-CRUD-Funktionalität, monolithisches Schema, noch ohne Authentifizierung und ohne Werthilfe.
v2.0.0	Juni 2024	Full-Stack & OData: Einführung von NextAuth.js, Google OAuth Integration, PostgreSQL 16, OData v4 Unterstützung, PLZ-Werthilfe und SMTP-E-Mail-Benachrichtigung.
v2.1.0	September 2026	Clean Code Edition: Vollständige Umsetzung aller 14 Befunde der Clean-Code-Analyse (B-01 bis B-08, F-01 bis F-06): <code>DbSeeder</code> ausgelagert, generischer <code>ODataCrudController<T></code> , <code>INotificationService</code> , Culture-Safe Vergleiche, durchgehend Controlled Inputs, <code>ConfirmDialog</code> und Type Augmentation.
v2.2.0	September 2026	Security & Structure Release: End-to-End JWT Bearer Authentifizierung (ASP.NET Core JwtBearer + NextAuth Session Token Forwarding), standardisierte Projektverzeichnisstruktur (<code>documentation/</code> , <code>scripts/</code> , <code>archive/</code>) und neues Handbuch.

Anhang A: Nützliche Links & Referenzen

Für vertiefte Studien und weiterführende Recherchen empfehlen wir die offiziellen Dokumentationen und Spezifikationen der eingesetzten Technologien:

Tabelle A.1 Offizielle Referenzen und Spezifikationen

Themenbereich	Ressource	URL / Link
OData v4	OASIS Standard Spezifikation & Tutorial	https://www.odata.org
JWT (RFC 7519)	Interaktiver Token Debugger & Spezifikation	https://jwt.io
Next.js 14	App Router Dokumentation & Best Practices	https://nextjs.org/docs
NextAuth.js	Authentifizierung für Next.js (Auth.js)	https://next-auth.js.org
ASP.NET Core JWT	Microsoft Learn: JWT-Bearer-Authentifizierung	learn.microsoft.com/.../jwt-authn
Entity Framework Core	EF Core Dokumentation & Npgsql Provider	https://learn.microsoft.com/ef/core
PostgreSQL	PostgreSQL 16 Handbuch & SQL-Referenz	https://www.postgresql.org/docs/16/
swisstopo AMTOVZ	Amtliches Schweizer Ortschaftsverzeichnis (CSV)	https://www.swisstopo.admin.ch

Abschlussbemerkung für Lernende

Mit dem Abschluss dieses Tutorials haben Sie eine vollständige, moderne und sichere Unternehmensanwendung von Grund auf verstanden. Die Kombination aus sauber getrennter Frontend- und Backend-Architektur, standardisierten Schnittstellen (OData), robuster Zustandslosigkeit (JWT) und automatisierten Tests/Builds (Docker) bildet das Rüstzeug für Ihre weitere berufliche Laufbahn als Informatiker/in EFZ Applikationsentwicklung.